LONDON METROPOLITAN UNIVERSITY

ITAHARI INTERNATIONAL COLLEGE

Module Code & Module Title

CC6012NT Data and Web Development

40% Individual Coursework

Submission: Milestone 2

Academic Semester: Autumn Semester 2025

Credit: 30 credit yearlong module

Student Name: Karuna Giri

London Met ID: 23049261

College ID: np05cp4a04230127

Assignment Submission Date: March 9, 2026

Assignment Due Date: March 9, 2026

Submitted To: Mr. Binay Koirala

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Table of Contents

1 Introduction 1

2 Tools and Technologies Used 2

2.1 Oracle SQL Developer 2

3 Initial Database Design 3

4 Database Normalization 4

4.1 UNF 5

4.2 1NF 6

4.3 2NF 7

4.4 3NF 9

5 Final Database Design 11

6 SQL Implementation and Query Results 12

6.1 Creating Tables 12

6.2 Inserting Query 21

7 Test Cases 30

7.1 Test Case for Basic Webform 30

7.2 Test Case for Complex Forms 46

8 Conclusion 50

9 References 51

Table of Figures

Figure 1: Figure of Oracle SQL Developer (oracle, 2025) 2
Figure 2: Figure of Initial ERD 3
Figure 3: Figure of Final ERD 11
Figure 4: Creating Users Table 12
Figure 5: Query Result of creating Users Table 12
Figure 6: Creating User_Booking Table 13
Figure 7: Query Result of creating User_Booking Table 13
Figure 8: Creating Booking Table 14
Figure 9: Query result of creating Booking Table 14
Figure 10: Creating Show Table 15
Figure 11: Query Result of creating Show Table 15
Figure 12: Creating Ticket_Booking Table 16
Figure 13: Query Result of creating Ticket_Booking Table 16
Figure 14: Creating Ticket Table 17
Figure 15: Query Result of creating Ticket Table 17
Figure 16: Creating Movie Table 18
Figure 17: Query Result of creating Movie Table 18
Figure 18: Creating Hall Table 19
Figure 19: Query Result of creating Hall Table 19
Figure 20: Creating Theater Table 20
Figure 21: Query Result Theater Table 20
Figure 22: Inserting data in Users Table 21
Figure 23: Query Result of inserting Uses Table 21
Figure 24: Inserting data in inserting User_Booking Table 22
Figure 25: Query Result of inserting User_Booking Table 22
Figure 26: Inserting data in Booking Table 23
Figure 27: Query Result of inserting Booking Table 23
Figure 28: Inserting data into Show Table 24
Figure 29: Query Result of inserting Show Table 24
Figure 30: Inserting data in Ticket_Booking Table 25

Figure 31: Query Result of inserting Ticket_Booking Table 25
Figure 32: Inserting data in Ticket Table 26
Figure 33: Query Results of inserting Ticket Table 26
Figure 34: Inserting data in Movie Table 27
Figure 35: Query Result of inserting Movie Table 27
Figure 36: Inserting data in Hall Table 28
Figure 37: Query Result of inserting Hall Table 28
Figure 38: Inserting data in Theater Table 29
Figure 39: Query Result of inserting Theater Table 29
Figure 40: Inserting new User Details 30
Figure 41: Result of Successfully added in the User Table 31
Figure 42: Updating exiting user details 32
Figure 43: Successfully updated in the user table 32
Figure 44: Successfully removed form user table 33
Figure 45: Inserting new Theater Details 34
Figure 46: Result of Successfully added in the Theater Table 35
Figure 47: Updating exiting theatre details 36
Figure 48: Successfully updated in the theatre table 36
Figure 49: Successfully removed form theatre table 37
Figure 50: Inserting new Movie Details 38
Figure 51: Result of Successfully added in the Movie Table 39
Figure 52: Updating exiting movie details 39
Figure 53: Successfully updated in the movie table 40
Figure 54: Successfully removed form movie table 41
Figure 55: Inserting new showtime Details 42
Figure 56: Result of Successfully added in the Showtime Table 43
Figure 57: Updating exiting showtime details 43
Figure 58: Successfully updated in the showtime table 44
Figure 59: Successfully removed form showtime table 45
Figure 60: Figure of User Tickets 47
Figure 61: Figure of Theater Movie 48

Figure 62: Figure of Movie TheatreCityHall Occupancy performer 49

Table of Tables

Table 1: Table 1: Test case for inserting new user 30
Table 2: Test Case of updating user details 31
Table 3: Test Case of deleting user 32
Table 4: Test case for inserting new theatre data 34
Table 5: Test Case of updating theatre details 35
Table 6: Test Case of deleting theatre data 36
Table 7: Test case for inserting new movie data 38
Table 8: Test Case of updating movie details 39
Table 9: Test Case of deleting movie data 40
Table 10: Test case for inserting new ShowTime data 42
Table 11: Test Case of updating showtime details 43
Table 12: Test Case of deleting showtime data 44
Table 13: Test Case of User Tickets 46
Table 14: Test Case of TheaterHallCity Movie 48
Table 15: Test Case of Movie TheatreCityHallOccupancy Performer 49

1 Introduction

Digital technologies have grown very fast, and this aspect has very much changed the manner at which organizations handle data and provide services. The successful utilization of movies, show times, ticket reservations and customer data is something that is required in the entertainment sector especially in the cinema houses to make operational effectiveness and good customer experience. An effective database system is important in guaranteeing the accuracy, consistency and availability of data and aiding in business decisions.

In this coursework, I am going to use a case study of KumariCinemas, a chain of operation of movies, which is located at various places in Nepal. The organization will use the introduction of centralized, automated, movie ticketing system to address the current challenges in the operational processes including ineffective ticket management, lack of communication and customer inconsistency among the halls within the cinema. The suggested system will facilitate movie scheduling, management of movie theaters halls, purchasing tickets via the system and then buying, price variation and cancellation.

Milestone 1 is aimed at analyzing the provided unnormalized data and to design a strong relational database system which meets the business needs of KumariCinemas. This milestone is interested in the conceptual and logical design of the database involving the creation of initial Entity Relationship (ER) diagram, systematic database normalization to the Third Normal Form (3NF) and the creation of final ER diagram based on the normalized schema. Also, in the milestone will entail the introduction of the database through the use of Oracle SQL, population of the data and running of the concerned SQL queries.

Through this milestone, the groundwork has been laid with regard to the creation of a web-based database application at later stages during the course work. The results of the given phase guarantee that the data is not redundant and will be reliable, and that the database structure will be scalable and will be able to meet the operational requirements of a modern cinema ticketing system.

2 Tools and Technologies Used

2.1 Oracle SQL Developer

Oracle SQL Developer is a free Oracle-based integrated development environment that is offered to develop, manage and test Oracle databases. It gives a graphical interface which is easy to use and enables users to write and execute SQL queries, create and modify database tables, as well as manage schema objects effectively. The tool facilitates Data Definition Language (DDL), Data Manipulation Language (DML) operations where the users can perform to create a database structure and fill it with sample data. Oracle SQL developer also enables the results of a query to be shown in a structured grid format and therefore it is easier to ensure the accuracy and integrity of the information. Other characteristics like being able to export query results, and capture execution results render it appropriate in documentation in academics. The Oracle SQL Developer was employed in the KumariCinemas project to construct tables according to the normalized schema, input test data, run the necessary SQL queries and store well-labeled screen shots of query outputs to be used in the submission of the Milestone 1. (oracle, 2025)

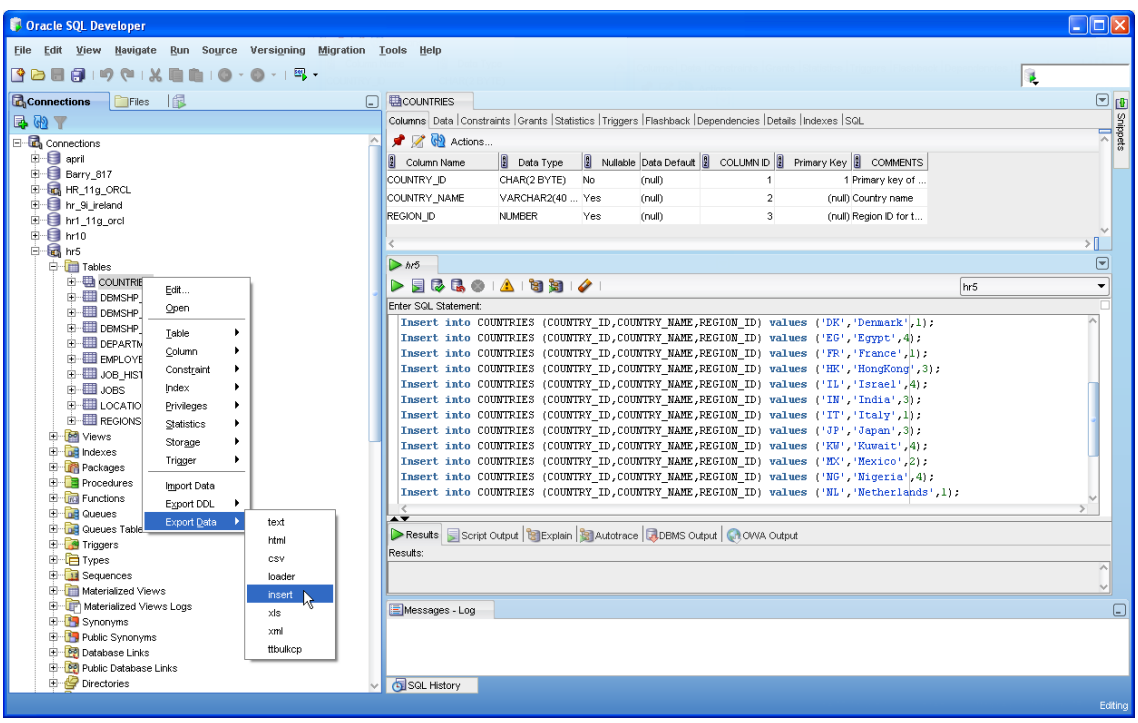


Figure 1: Figure of Oracle SQL Developer (oracle, 2025)

3 Initial Database Design

Initial database design refers to the practice of structuring and putting data requirements into a conceptual model prior to any actual work being done in technical implementation. As per the basic rules of database design, this step entails determining the entities, attributes and relationships that are pertinent to the area of problem, and applying tools such as entity relationship diagrams (ERDs) to present this structure. The design forms a basis on which one can refine and normalize later and to understand the dependencies and cardinalities of data in a high level.

The unnormalized data of cinema ticketing, which comprised movies, users and theatres, halls, showtime and tickets, was analyzed in the context of the KumariCinemas coursework to define the candidate entities and the relationships among them. This analysis led to the creation of an initial ERD that shows the conceptual structure of the database before the use of normalization techniques to enhance the integrity of the data and eliminate redundancy. (lucidchart, 2025)

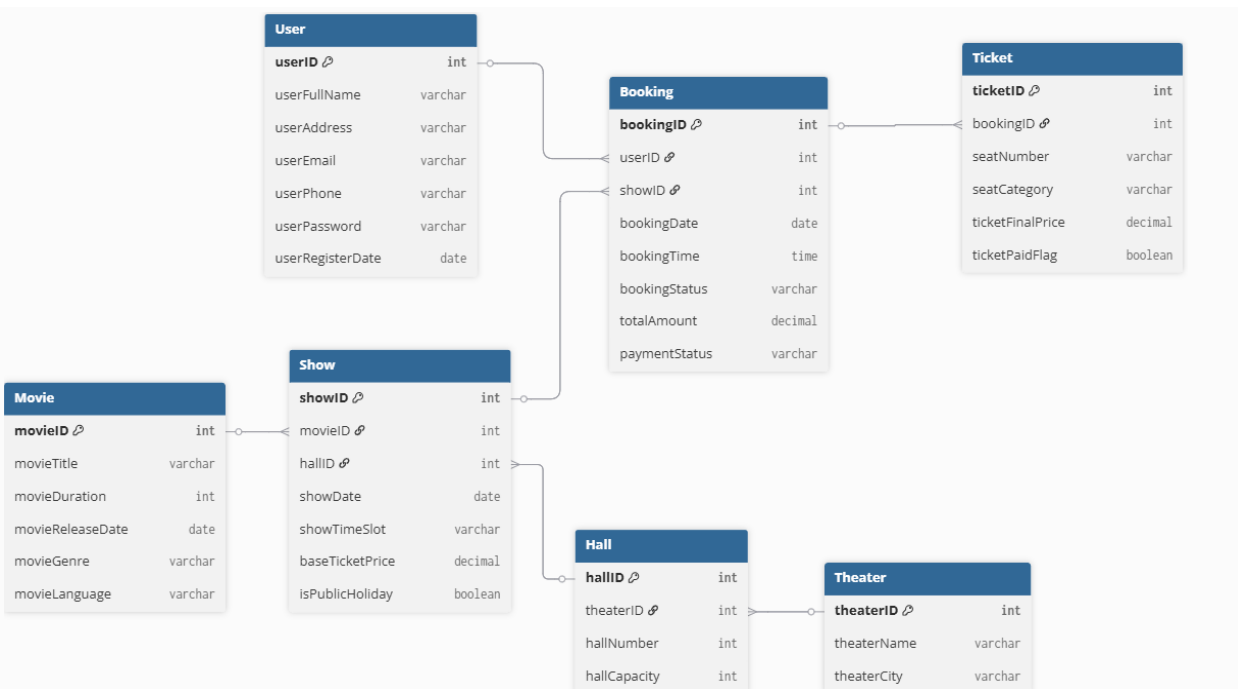


Figure 2: Figure of Initial ERD

4 Database Normalization

Database normalization is a relational database design technique used to regulate data tables in a rational and effective way by minimizing the repetitions and enhancing the integrity of data. It is the breaking down of the large and unstructured tables into small and well-structured ones in such a manner that each table will have only related information and the relevance of the tables is provided by the use of appropriate keys. Normalization is most commonly done in multiple steps or forms of normalization (1NF, 2NF, and 3NF) dealing with particular forms of redundancy and dependency problems. These rules enable the end database design to be more reflective of information and relationship, reduce data duplication and decrease the chances of update, insertion and deletion anomaly. (Dancuk, 2021)

4.1 UNF

The first stage of an unnormalized data is called Unnormalized Form (UNF), which refers to a dataset that has not been transformed by any normalization rule. Within this format, it is possible to use a database table with repeating groups, multi-valued attributes and complex and non-atomic values, which are nonconformist to fundamental relational ideas. Due to the inability to satisfy the conditions of the First Normal Form, the information represented in UNF tables is often highly redundant and inconsistent. An example of this would be that one field could store several values or lists hence it would be hard to update, insert or delete records without some errors. UNF is the initial step in the normalization of the database and the goal is to refine and partition the raw table into well-structured relations that remove such problems and enhance the integrity of the data.

The Unnormalized Form of E-Classroom Platform is described as below:

Kumari Cinema (<u>userID</u>, userFullName, userAddress, userEmail, userPhone, userPassword, userRegisterDate, {bookingID, bookingDate, bookingTime, bookingStatus, totalAmount, paymentStatus, showID, showDate, showTimeSlot, baseTicketPrice, isPublicHoliday, {ticketID, seatNumber, seatCategory, ticketFinalPrice, ticketPaidFlag, movieID, movieTitle, movieDuration, movieReleaseDate, movieGenre, movieLanguage, hallID, hallNumber, hallCapacity, theaterID, theaterName, theaterCity}})

The UNF table corresponds to the Kumari Cinema ticketing object in which all details associated with users, bookings, shows, tickets, movies, halls, and theaters are loaded into a single unnormalized structure. It is possible to have multiple bookings to a particular user and also within a particular booking many tickets, indicated by repeat and nested groups. The details of cinema, hall and theatre are duplicated per ticket and therefore there is a semblance of a data duplication. The design predisposes the anomaly of update, insertion and deletion, and makes the data hard to handle effectively. As such, the table fails to meet the criteria of normalization and it must be broken into related tables.

4.2 1NF

First Normal Form states that all the values in a table are made up of atomic (indivisible) values, that is every field of the table holds only one value not a set or a list of values. Tables in 1NF do not contain repetitive groups or nested databases and every row is uniquely identifiable, usually using a primary key. A table can be converted to 1NF, a process that can be done through decomposing composite or multi-valued columns into rows or related tables as a way of ensuring that a column contains a single piece of information. To gain a normalization process the first stage is the 1NF, a stage that makes sure the data is in a tabular format that is uniform and then the process can be further refined. (Dancuk, 2021)

The First Normalized Form of the KumariCinemas is discussed as below:

User_1 (<u>userID</u>, userFullName, userAddress, userEmail, userPhone, userPassword, userRegisterDate)

Booking_1 (<u>bookingID</u>, bookingDate, bookingTime, bookingStatus, totalAmount, paymentStatus, showID, showDate, showTimeSlot, baseTicketPrice, isPublicHoliday, <u>userID*</u>)

Ticket_1 (<u>ticketID</u>, seatNumber, seatCategory, ticketFinalPrice, ticketPaidFlag, movieID, movieTitle, movieDuration, movieReleaseDate, movieGenre, movieLanguage, hallID, hallNumber, hallCapacity, theaterID, theaterName, theaterCity, <u>bookingID*</u>)

The unnormalized Kumari Cinema data is formatted into the First Normal Form (1NF): repetitions and self-nesting groups are eliminated and all the attributes are represented by atomic (indivisible) values. The information of the users is divided to form the User_1 table with one record/user. The Booking-1 table contains the booking related information, such as the details of the show and is connected as a foreign key with the users. Ticket is stored in Ticket 1 table with each ticket showing one seat and is associated with a booking. Even though there is still the redundancy of data in the form of movie, hall and theater attributes, the structure is now compliant with 1NF principles by removing multi valued attributes and ensuring that fields have a single value.

4.3 2NF

Second Normal Form is an extension of the 1NF that gets rid of partial dependency. A table is in 2NF provided that it is already in 1NF and that all the non-key attributes are not dependent on a portion of the key, but rather on the full key. Partial dependency is the dependency on a non-key attribute by some section of a composite key. In reality, to achieve 2NF, tables are usually divided to ensure that the attributes are totally functionally dependent on the entire primary key to eliminate redundancy and enhance data organization. (Dancuk, 2021)

The Second Normal Form of KumariCinemas is described as below:

User_2 (<u>userID</u>, userFullName, userAddress, userEmail, userPhone, userPassword, userRegisterDate)

User_Booking_2 (<u>bookingID*</u>, <u>userID*</u>)

Booking_2 (<u>bookingID</u>, bookingDate, bookingTime, bookingStatus, totalAmount, paymentStatus, showID, showDate, showTimeSlot, baseTicketPrice, isPublicHoliday)

Ticket_Booking_2 (<u>ticketID*</u>, <u>bookingID*</u>)

Ticket_2 (<u>ticketID</u>, seatNumber, seatCategory, ticketFinalPrice, ticketPaidFlag, movieID, movieTitle, movieDuration, movieReleaseDate, movieGenre, movieLanguage, hallID, hallNumber, hallCapacity, theaterID, theaterName, theaterCity)

The Second Normal Form (2NF) is another form of the design that removes partial dependencies that were present in the 1NF. User information is stored in the User2 table and relationship between user and bookings is divided into the User Booking 2 bridge table whereby the non-key attributes do not depend on the entire major key. Attributes that are related to booking are transferred to the Booking_2 table to ensure that all the booking information is based on just one attribute, which is the bookingID. In the same way, the relationship between bookings and tickets, which is many to many is solved with Ticket_Booking_2 bridge table and ticket information are stored in Ticket_2 table. This

design makes sure that all non-key attributes are fully functionally dependent on the primary key of its table and thus meet the demands of Second Normal Form.

4.4 3NF

Third Normal Form is another type of database structure where transitive dependencies are eliminated. A table is 3NF provided that it is already 2NF and the non key attribute has no dependencies on another non key attribute. That is, none of the non-key data should have a direct dependence with anything other than the primary key. The conversion of a table to 3NF typically includes splitting the table into other tables in order to remove the indirect relationships in the table thereby improving data integrity and allowing effective updates and insertions of data along with their removal. (Dancuk, 2021)

The Third Normal Form of KumariCinemas is described as below:

User_3 (<u>userID</u>, userFullName, userAddress, userEmail, userPhone, userPassword, userRegisterDate)

User_Booking_3 (<u>bookingID*</u>, <u>userID*</u>)

Booking_3 (<u>bookingID</u>, bookingDate, bookingTime, bookingStatus, totalAmount, paymentStatus)

Show_3 (<u>showID</u>, showDate, showTimeSlot, baseTicketPrice, isPublicHoliday, <u>bookingID*</u>)

Ticket_Booking_3 (<u>ticketID*</u>, <u>bookingID*</u>)

Ticket_3 (<u>ticketID</u>, seatNumber, seatCategory, ticketFinalPrice, ticketPaidFlag, <u>bookingID*</u>)

Movie_3 (<u>movieID</u>, movieTitle, movieDuration, movieReleaseDate, movieGenre, movieLanguage, <u>ticketID*</u>)

Hall_3 (<u>hallID</u>, hallNumber, hallCapacity, <u>ticketID*</u>)

Theater_3 (<u>theaterID</u>, theaterName, theaterCity, <u>ticketID*</u>)

The database design is completed in the Third normal form (3NF) by eliminating transitive dependencies and making sure that the non-key attributes of the design have their dependencies neither on more than the primary key nor on other values. The information about users is stored separately in the table User_3 whereas the relationships between the user and the booking are maintained in the bridge table User_Booking_3. The booking information has been split into Booking 3 and the show information has to be transferred to Show 3 table so that the booking and show attributes do not have dependencies with each other. Information on the tickets is stored in Ticket_3 and connected with the booking through Ticket_Booking_3. Details of movies, halls and theatres are stored in distinct tables (Movie_3, Hall_3 and Theater_3) which do not contain any redundancy and/or transitive relationships. The structure is completely 3NF as it offers a highly structured, scalable, and anomaly-free database design to the Kumari Cinema system.

5 Final Database Design

The Final Database Design is the final logical structure of the KumariCinemas database once all the normalization principles are implemented and the schema is implemented. This phase gives database design principles where data is separated and placed in clear-cut tables which meet the normal forms and excludes redundancy and unwanted dependencies to enhance the data integrity and minimize the likelihood of anomalies in data in case it is altered, added or removed. (Alice Gomstyn, 2025) In the final design, the primary keys are well defined, the atomic attributes will be present and there will be a proper foreign key connection between the two or more tables that are supposed to be linked. Practically speaking, to do this coursework, the completed database is represented in the final ERD which can be easily converted into SQL CREATE TABLE statements to generate a highly effective and efficient relational database to support the KumariCinemas ticketing system.

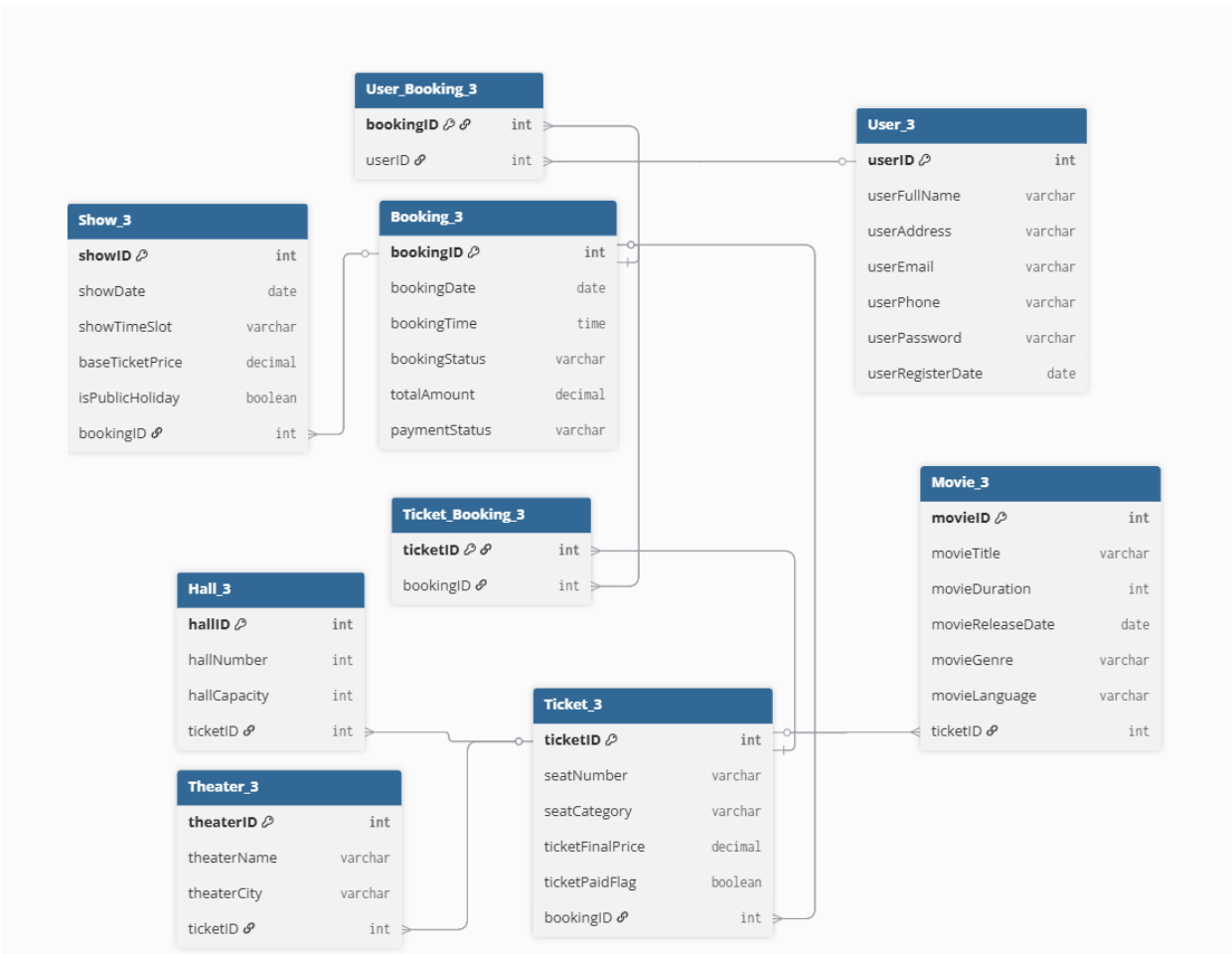


Figure 3: Figure of Final ERD

6 SQL Implementation and Query Results

The SQL Implementation of final database design of KumariCinemas is implemented in Oracle SQL Developer which is mentioned below:

6.1 Creating Tables

- **Users Table**

```sql
// User Table
CREATE TABLE Users (
    userID NUMBER PRIMARY KEY,
    userFullName VARCHAR2(100) NOT NULL,
    userAddress VARCHAR2(200),
    userEmail VARCHAR2(100) UNIQUE NOT NULL,
    userPhone VARCHAR2(20),
    userPassword VARCHAR2(100) NOT NULL,
    userRegisterDate DATE DEFAULT SYSDATE
);
```

Figure 4: Creating Users Table

Columns | Data | Model | Constraints | Grants | Statistics | Triggers | Flashback | Dependencies | Details | Partitions | Indexes | SQL

Actions...

	COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	USERID	NUMBER	No	(null)	1	(null)
2	USERFULLNAME	VARCHAR2(100 BYTE)	No	(null)	2	(null)
3	USERADDRESS	VARCHAR2(200 BYTE)	Yes	(null)	3	(null)
4	USEREMAIL	VARCHAR2(100 BYTE)	No	(null)	4	(null)
5	USERPHONE	VARCHAR2(20 BYTE)	Yes	(null)	5	(null)
6	USERPASSWORD	VARCHAR2(100 BYTE)	No	(null)	6	(null)
7	USERREGISTERDATE	DATE	Yes	SYSDATE	7	(null)

Figure 5: Query Result of creating Users Table

- User_Booking Table

```
// User_Booking Table
CREATE TABLE User_Booking (
    bookingID NUMBER PRIMARY KEY,
    userID NUMBER NOT NULL,
    CONSTRAINT fk_ub_booking
        FOREIGN KEY (bookingID) REFERENCES Booking(bookingID),
    CONSTRAINT fk_ub_user
        FOREIGN KEY (userID) REFERENCES Users(userID)
);
```

Figure 6: Creating User_Booking Table

Columns | Data | Model | Constraints | Grants | Statistics | Triggers | Flashback | Dependencies | Details | Partitions | Indexes | SQL

Actions...

	COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	TICKETID	NUMBER	No	(null)	1	(null)
2	BOOKINGID	NUMBER	No	(null)	2	(null)

Figure 7: Query Result of creating User_Booking Table

- Booking Table

```sql
// Booking Table
CREATE TABLE Booking (
    bookingID NUMBER PRIMARY KEY,
    bookingDate DATE NOT NULL,
    bookingTime VARCHAR2(20),
    bookingStatus VARCHAR2(30),
    totalAmount NUMBER(10,2),
    paymentStatus VARCHAR2(30)
);
```

Figure 8: Creating Booking Table

Columns | Data | Model | Constraints | Grants | Statistics | Triggers | Flashback | Dependencies | Details | Partitions | Indexes | SQL

Actions...

	COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	BOOKINGID	NUMBER	No	(null)	1	(null)
2	BOOKINGDATE	DATE	No	(null)	2	(null)
3	BOOKINGTIME	VARCHAR2(20 BYTE)	Yes	(null)	3	(null)
4	BOOKINGSTATUS	VARCHAR2(30 BYTE)	Yes	(null)	4	(null)
5	TOTALAMOUNT	NUMBER(10,2)	Yes	(null)	5	(null)
6	PAYMENTSTATUS	VARCHAR2(30 BYTE)	Yes	(null)	6	(null)

Figure 9: Query result of creating Booking Table

- Show Table

```sql
// Show Table
CREATE TABLE Show (
    showID NUMBER PRIMARY KEY,
    showDate DATE NOT NULL,
    showTimeSlot VARCHAR2(20),
    baseTicketPrice NUMBER(8,2),
    isPublicHoliday CHAR(1) CHECK (isPublicHoliday IN ('Y','N')),
    bookingID NUMBER,
    CONSTRAINT fk_show_booking
        FOREIGN KEY (bookingID) REFERENCES Booking(bookingID)
);
```

Figure 10: Creating Show Table

Coursework.sql | Data_Coursework | SHOW

Columns | Data | Model | Constraints | Grants | Statistics | Triggers | Flashback | Dependencies | Details | Partitions | Indexes | SQL

Actions...

	COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	SHOWID	NUMBER	No	(null)	1	(null)
2	SHOWDATE	DATE	No	(null)	2	(null)
3	SHOWTIMESLOT	VARCHAR2(20 BYTE)	Yes	(null)	3	(null)
4	BASETICKETPRICE	NUMBER(8,2)	Yes	(null)	4	(null)
5	ISPUBLICHOLIDAY	CHAR(1 BYTE)	Yes	(null)	5	(null)
6	BOOKINGID	NUMBER	Yes	(null)	6	(null)

Figure 11: Query Result of creating Show Table

- Ticket_Booking Table

```
// Ticket_Booking Table
CREATE TABLE Ticket_Booking (
    ticketID NUMBER PRIMARY KEY,
    bookingID NUMBER NOT NULL,
    CONSTRAINT fk_tb_ticket
        FOREIGN KEY (ticketID) REFERENCES Ticket(ticketID),
    CONSTRAINT fk_tb_booking
        FOREIGN KEY (bookingID) REFERENCES Booking(bookingID)
);
```

Figure 12: Creating Ticket_Booking Table

Columns | Data | Model | Constraints | Grants | Statistics | Triggers | Flashback | Dependencies | Details | Partitions | Indexes | SQL

Actions...

	COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	TICKETID	NUMBER	No	(null)	1	(null)
2	BOOKINGID	NUMBER	No	(null)	2	(null)

Figure 13: Query Result of creating Ticket_Booking Table

- Ticket Table

```sql
// Ticket Table
CREATE TABLE Ticket (
    ticketID NUMBER PRIMARY KEY,
    seatNumber VARCHAR2(10),
    seatCategory VARCHAR2(30),
    ticketFinalPrice NUMBER(8,2),
    ticketPaidFlag CHAR(1) CHECK (ticketPaidFlag IN ('Y','N')),
    bookingID NUMBER,
    CONSTRAINT fk_ticket_booking
        FOREIGN KEY (bookingID) REFERENCES Booking(bookingID)
);
```

Figure 14: Creating Ticket Table

Columns | Data | Model | Constraints | Grants | Statistics | Triggers | Flashback | Dependencies | Details | Partitions | Indexes | SQL

Actions...

	COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	TICKETID	NUMBER	No	(null)	1	(null)
2	SEATNUMBER	VARCHAR2(10 BYTE)	Yes	(null)	2	(null)
3	SEATCATEGORY	VARCHAR2(30 BYTE)	Yes	(null)	3	(null)
4	TICKETFINALPRICE	NUMBER(8,2)	Yes	(null)	4	(null)
5	TICKETPAIDFLAG	CHAR(1 BYTE)	Yes	(null)	5	(null)
6	BOOKINGID	NUMBER	Yes	(null)	6	(null)

Figure 15: Query Result of creating Ticket Table

- Movie Table

```sql
// Movie Table
CREATE TABLE Movie (
    movieID NUMBER PRIMARY KEY,
    movieTitle VARCHAR2(150) NOT NULL,
    movieDuration NUMBER,
    movieReleaseDate DATE,
    movieGenre VARCHAR2(50),
    movieLanguage VARCHAR2(50),
    ticketID NUMBER,
    CONSTRAINT fk_movie_ticket
        FOREIGN KEY (ticketID) REFERENCES Ticket(ticketID)
);
```

Figure 16: Creating Movie Table

Columns | Data | Model | Constraints | Grants | Statistics | Triggers | Flashback | Dependencies | Details | Partitions | Indexes | SQL

Actions...

	COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	MOVIEID	NUMBER	No	(null)	1	(null)
2	MOVIETITLE	VARCHAR2(150 BYTE)	No	(null)	2	(null)
3	MOVIEDURATION	NUMBER	Yes	(null)	3	(null)
4	MOVIERELEASEDATE	DATE	Yes	(null)	4	(null)
5	MOVIEGENRE	VARCHAR2(50 BYTE)	Yes	(null)	5	(null)
6	MOVIELANGUAGE	VARCHAR2(50 BYTE)	Yes	(null)	6	(null)
7	TICKETID	NUMBER	Yes	(null)	7	(null)

Figure 17: Query Result of creating Movie Table

- Hall Table

```
// Hall Table
CREATE TABLE Hall (
    hallID NUMBER PRIMARY KEY,
    hallNumber NUMBER,
    hallCapacity NUMBER,
    ticketID NUMBER,
    CONSTRAINT fk_hall_ticket
        FOREIGN KEY (ticketID) REFERENCES Ticket(ticketID)
);
```

Figure 18: Creating Hall Table

Columns | Data | Model | Constraints | Grants | Statistics | Triggers | Flashback | Dependencies | Details | Partitions | Indexes | SQL

Actions...

	COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	HALLID	NUMBER	No	(null)	1	(null)
2	HALLNUMBER	NUMBER	Yes	(null)	2	(null)
3	HALLCAPACITY	NUMBER	Yes	(null)	3	(null)
4	TICKETID	NUMBER	Yes	(null)	4	(null)

Figure 19: Query Result of creating Hall Table

- Theater Table

```sql
// Theater Table
CREATE TABLE Theater (
    theaterID NUMBER PRIMARY KEY,
    theaterName VARCHAR2(100) NOT NULL,
    theaterCity VARCHAR2(100),
    ticketID NUMBER,
    CONSTRAINT fk_theater_ticket
        FOREIGN KEY (ticketID) REFERENCES Ticket(ticketID)
);
```

Figure 20: Creating Theater Table

Columns | Data | Model | Constraints | Grants | Statistics | Triggers | Flashback | Dependencies | Details | Partitions | Indexes | SQL

Actions...

	COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	THEATERID	NUMBER	No	(null)	1	(null)
2	THEATERNAME	VARCHAR2(100 BYTE)	No	(null)	2	(null)
3	THEATERCITY	VARCHAR2(100 BYTE)	Yes	(null)	3	(null)
4	TICKETID	NUMBER	Yes	(null)	4	(null)

Figure 21: Query Result Theater Table

6.2 Inserting Query

- Users Table

```
// User Table

INSERT INTO Users VALUES (1,'Prakash Joshi','Baneshwor, Kathmandu','prakash.j@gmail.com','981111001','pwd001',SYSDATE-160);
INSERT INTO Users VALUES (2,'Nirmala Basnet','New Road, Pokhara','nirmala.b@gmail.com','981111002','pwd002',SYSDATE-155);
INSERT INTO Users VALUES (3,'Sanjay Mishra','Main Road, Janakpur','sanjay.m@gmail.com','981111003','pwd003',SYSDATE-150);
INSERT INTO Users VALUES (4,'Rekha Poudel','Traffic Chowk, Butwal','rekha.p@gmail.com','981111004','pwd004',SYSDATE-145);
INSERT INTO Users VALUES (5,'Dipesh Shahi','Surkhet Bazaar','dipesh.s@gmail.com','981111005','pwd005',SYSDATE-140);
INSERT INTO Users VALUES (6,'Kalpana Magar','Tansen, Palpa','kalpana.m@gmail.com','981111006','pwd006',SYSDATE-135);
INSERT INTO Users VALUES (7,'Rohit Yadav','Rajbiraj','rohit.y@gmail.com','981111007','pwd007',SYSDATE-130);
INSERT INTO Users VALUES (8,'Sabina Karki','Damak, Jhapa','sabina.k@gmail.com','981111008','pwd008',SYSDATE-125);
INSERT INTO Users VALUES (9,'Ashok Chaudhary','Tikapur, Kailali','ashok.c@gmail.com','981111009','pwd009',SYSDATE-120);
INSERT INTO Users VALUES (10,'Meena Thapa','Gaur, Rautahat','meena.t@gmail.com','981111010','pwd010',SYSDATE-115);
```

Figure 22: Inserting data in Users Table

Columns | Data | Model | Constraints | Grants | Statistics | Triggers | Flashback | Dependencies | Details | Partitions | Indexes | SQL

Sort.. | Filter:

	USERID	USERFULLNAME	USERADDRESS	USEREMAIL	USERPHONE	USERPASSWORD	USERREGISTERDATE
1	1	Prakash Joshi	Baneshwor, Kathmandu	prakash.j@gmail.com	981111001	pwd001	16-07-25
2	2	Nirmala Basnet	New Road, Pokhara	nirmala.b@gmail.com	981111002	pwd002	21-07-25
3	3	Sanjay Mishra	Main Road, Janakpur	sanjay.m@gmail.com	981111003	pwd003	26-07-25
4	4	Rekha Poudel	Traffic Chowk, Butwal	rekha.p@gmail.com	981111004	pwd004	31-07-25
5	5	Dipesh Shahi	Surkhet Bazaar	dipesh.s@gmail.com	981111005	pwd005	05-08-25
6	6	Kalpana Magar	Tansen, Palpa	kalpana.m@gmail.com	981111006	pwd006	10-08-25
7	7	Rohit Yadav	Rajbiraj	rohit.y@gmail.com	981111007	pwd007	15-08-25
8	8	Sabina Karki	Damak, Jhapa	sabina.k@gmail.com	981111008	pwd008	20-08-25
9	9	Ashok Chaudhary	Tikapur, Kailali	ashok.c@gmail.com	981111009	pwd009	25-08-25
10	10	Meena Thapa	Gaur, Rautahat	meena.t@gmail.com	981111010	pwd010	30-08-25

Figure 23: Query Result of inserting Uses Table

- User_Booking Table

```
// Users_Booking Table

INSERT INTO User_Booking VALUES (1,4);
INSERT INTO User_Booking VALUES (2,1);
INSERT INTO User_Booking VALUES (3,7);
INSERT INTO User_Booking VALUES (4,3);
INSERT INTO User_Booking VALUES (5,9);
INSERT INTO User_Booking VALUES (6,2);
INSERT INTO User_Booking VALUES (7,10);
INSERT INTO User_Booking VALUES (8,5);
INSERT INTO User_Booking VALUES (9,6);
INSERT INTO User_Booking VALUES (10,8);
```

Figure 24: Inserting data in inserting User_Booking Table

Columns | Data | Model | Constraints | Grants | Statistics

Sort.. | Filter:

	BOOKINGID	USERID
1	1	4
2	2	1
3	3	7
4	4	3
5	5	9
6	6	2
7	7	10
8	8	5
9	9	6
10	10	8

Figure 25: Query Result of inserting User_Booking Table

- Booking Table

```
// Booking Table

INSERT INTO Booking VALUES (1,SYSDATE-25,'09:30 AM','CONFIRMED',1350,'PAID');
INSERT INTO Booking VALUES (2,SYSDATE-35,'11:00 AM','CONFIRMED',1650,'PAID');
INSERT INTO Booking VALUES (3,SYSDATE-45,'02:00 PM','CONFIRMED',1850,'PAID');
INSERT INTO Booking VALUES (4,SYSDATE-55,'04:30 PM','CONFIRMED',2100,'PAID');
INSERT INTO Booking VALUES (5,SYSDATE-65,'07:00 PM','CONFIRMED',2250,'PAID');
INSERT INTO Booking VALUES (6,SYSDATE-75,'10:15 AM','CONFIRMED',1450,'PAID');
INSERT INTO Booking VALUES (7,SYSDATE-85,'01:45 PM','CONFIRMED',1750,'PAID');
INSERT INTO Booking VALUES (8,SYSDATE-18,'05:30 PM','CONFIRMED',1950,'PAID');
INSERT INTO Booking VALUES (9,SYSDATE-12,'08:00 PM','CONFIRMED',2150,'PAID');
INSERT INTO Booking VALUES (10,SYSDATE-7,'06:15 PM','CONFIRMED',2350,'PAID');
```

Figure 26: Inserting data in Booking Table

Columns | Data | Model | Constraints | Grants | Statistics | Triggers | Flashback | Dependencies | Details | Partitions | Indexes | SQL

Sort.. | Filter:

	BOOKINGID	BOOKINGDATE	BOOKINGTIME	BOOKINGSTATUS	TOTALAMOUNT	PAYMENTSTATUS
1	1	28-11-25	09:30 AM	CONFIRMED	1350	PAID
2	2	18-11-25	11:00 AM	CONFIRMED	1650	PAID
3	3	08-11-25	02:00 PM	CONFIRMED	1850	PAID
4	4	29-10-25	04:30 PM	CONFIRMED	2100	PAID
5	5	19-10-25	07:00 PM	CONFIRMED	2250	PAID
6	6	09-10-25	10:15 AM	CONFIRMED	1450	PAID
7	7	29-09-25	01:45 PM	CONFIRMED	1750	PAID
8	8	05-12-25	05:30 PM	CONFIRMED	1950	PAID
9	9	11-12-25	08:00 PM	CONFIRMED	2150	PAID
10	10	16-12-25	06:15 PM	CONFIRMED	2350	PAID

Figure 27: Query Result of inserting Booking Table

- Show Table

```
// Show Table

INSERT INTO Show VALUES (1,SYSDATE-6,'Morning',330,'N',3);
INSERT INTO Show VALUES (2,SYSDATE-7,'Day',370,'N',1);
INSERT INTO Show VALUES (3,SYSDATE-8,'Evening',420,'Y',5);
INSERT INTO Show VALUES (4,SYSDATE-9,'Night',470,'Y',2);
INSERT INTO Show VALUES (5,SYSDATE-10,'Morning',350,'N',7);
INSERT INTO Show VALUES (6,SYSDATE-11,'Day',390,'N',4);
INSERT INTO Show VALUES (7,SYSDATE-12,'Evening',440,'Y',9);
INSERT INTO Show VALUES (8,SYSDATE-13,'Night',500,'Y',6);
INSERT INTO Show VALUES (9,SYSDATE-14,'Evening',460,'N',10);
INSERT INTO Show VALUES (10,SYSDATE-15,'Night',530,'Y',8);
```

Figure 28: Inserting data into Show Table

Columns | Data | Model | Constraints | Grants | Statistics | Triggers | Flashback | Dependencies | Details | Partitions | Indexes | SQL

Sort.. | Filter:

	SHOWID	SHOWDATE	SHOWTIMESLOT	BASETICKETPRICE	ISPUBLICHOLIDAY	BOOKINGID
1	1	17-12-25	Morning	330	N	3
2	2	16-12-25	Day	370	N	1
3	3	15-12-25	Evening	420	Y	5
4	4	14-12-25	Night	470	Y	2
5	5	13-12-25	Morning	350	N	7
6	6	12-12-25	Day	390	N	4
7	7	11-12-25	Evening	440	Y	9
8	8	10-12-25	Night	500	Y	6
9	9	09-12-25	Evening	460	N	10
10	10	08-12-25	Night	530	Y	8

Figure 29: Query Result of inserting Show Table

- Ticket_Booking Table

```
// Ticket_Booking Table

INSERT INTO Ticket_Booking VALUES (1,3);
INSERT INTO Ticket_Booking VALUES (2,1);
INSERT INTO Ticket_Booking VALUES (3,5);
INSERT INTO Ticket_Booking VALUES (4,2);
INSERT INTO Ticket_Booking VALUES (5,7);
INSERT INTO Ticket_Booking VALUES (6,4);
INSERT INTO Ticket_Booking VALUES (7,9);
INSERT INTO Ticket_Booking VALUES (8,6);
INSERT INTO Ticket_Booking VALUES (9,10);
INSERT INTO Ticket_Booking VALUES (10,8);
```

Figure 30: Inserting data in Ticket_Booking Table

Columns | Data | Model | Constraints | Grants | Statistics

Sort.. | Filter:

	TICKETID	BOOKINGID
1	1	3
2	2	1
3	3	5
4	4	2
5	5	7
6	6	4
7	7	9
8	8	6
9	9	10
10	10	8

Figure 31: Query Result of inserting Ticket_Booking Table

- Ticket Table

```
// Ticket Table

INSERT INTO Ticket VALUES (1,'F3','Gold',540,'Y',3);
INSERT INTO Ticket VALUES (2,'G5','Silver',410,'Y',1);
INSERT INTO Ticket VALUES (3,'H7','Gold',575,'Y',5);
INSERT INTO Ticket VALUES (4,'J2','Silver',430,'Y',2);
INSERT INTO Ticket VALUES (5,'K4','Gold',595,'Y',7);
INSERT INTO Ticket VALUES (6,'L6','Silver',405,'Y',4);
INSERT INTO Ticket VALUES (7,'M1','Gold',620,'Y',9);
INSERT INTO Ticket VALUES (8,'N8','Silver',480,'Y',6);
INSERT INTO Ticket VALUES (9,'P9','Gold',645,'Y',10);
INSERT INTO Ticket VALUES (10,'Q10','Silver',505,'Y',8);
```

Figure 32: Inserting data in Ticket Table

Columns | Data | Model | Constraints | Grants | Statistics | Triggers | Flashback | Dependencies | Details | Partitions | Indexes | SQL

Sort.. | Filter:

	TICKETID	SEATNUMBER	SEATCATEGORY	TICKETFINALPRICE	TICKETPAIDFLAG	BOOKINGID
1	1	F3	Gold	540	Y	3
2	2	G5	Silver	410	Y	1
3	3	H7	Gold	575	Y	5
4	4	J2	Silver	430	Y	2
5	5	K4	Gold	595	Y	7
6	6	L6	Silver	405	Y	4
7	7	M1	Gold	620	Y	9
8	8	N8	Silver	480	Y	6
9	9	P9	Gold	645	Y	10
10	10	Q10	Silver	505	Y	8

Figure 33: Query Results of inserting Ticket Table

- Movie Table

```
// Movie Table

INSERT INTO Movie VALUES (1,'Mahapurush',140,DATE '2022-10-14','Drama','Nepali',3);
INSERT INTO Movie VALUES (2,'Prem Geet 3',165,DATE '2022-09-23','Romance','Nepali',1);
INSERT INTO Movie VALUES (3,'The Kashmir Files',170,DATE '2022-03-11','Drama','Hindi',5);
INSERT INTO Movie VALUES (4,'Black Panther: Wakanda Forever',160,DATE '2022-11-11','Action','English',2);
INSERT INTO Movie VALUES (5,'Kantara',148,DATE '2022-09-30','Thriller','Kannada',7);
INSERT INTO Movie VALUES (6,'Brahmastra',167,DATE '2022-09-09','Fantasy','Hindi',4);
INSERT INTO Movie VALUES (7,'Sita Ramam',158,DATE '2022-08-05','Romance','Telugu',9);
INSERT INTO Movie VALUES (8,'Doctor Strange: Multiverse',150,DATE '2022-05-06','Sci-Fi','English',6);
INSERT INTO Movie VALUES (9,'KGF Chapter 1',155,DATE '2018-12-21','Action','Kannada',10);
INSERT INTO Movie VALUES (10,'Jawan',169,DATE '2023-09-07','Action','Hindi',8);
```

Figure 34: Inserting data in Movie Table

Columns | Data | Model | Constraints | Grants | Statistics | Triggers | Flashback | Dependencies | Details | Partitions | Indexes | SQL

Sort.. | Filter:

	MOVIEID	MOVIETITLE	MOVIEDURATION	MOVIERELEASEDATE	MOVIEGENRE	MOVIELANGUAGE	TICKETID
1	1	Mahapurush	140	14-10-22	Drama	Nepali	3
2	2	Prem Geet 3	165	23-09-22	Romance	Nepali	1
3	3	The Kashmir Files	170	11-03-22	Drama	Hindi	5
4	4	Black Panther: Wakanda Forever	160	11-11-22	Action	English	2
5	5	Kantara	148	30-09-22	Thriller	Kannada	7
6	6	Brahmastra	167	09-09-22	Fantasy	Hindi	4
7	7	Sita Ramam	158	05-08-22	Romance	Telugu	9
8	8	Doctor Strange: Multiverse	150	06-05-22	Sci-Fi	English	6
9	9	KGF Chapter 1	155	21-12-18	Action	Kannada	10
10	10	Jawan	169	07-09-23	Action	Hindi	8

Figure 35: Query Result of inserting Movie Table

- Hall Table

```
// Hall Table

INSERT INTO Hall VALUES (1,201,180,3);
INSERT INTO Hall VALUES (2,202,200,1);
INSERT INTO Hall VALUES (3,203,220,5);
INSERT INTO Hall VALUES (4,204,160,2);
INSERT INTO Hall VALUES (5,205,250,7);
INSERT INTO Hall VALUES (6,206,190,4);
INSERT INTO Hall VALUES (7,207,300,9);
INSERT INTO Hall VALUES (8,208,210,6);
INSERT INTO Hall VALUES (9,209,230,10);
INSERT INTO Hall VALUES (10,210,260,8);
```

Figure 36: Inserting data in Hall Table

Columns | Data | Model | Constraints | Grants | Statistics | Triggers | Flashback

Sort.. | Filter:

	HALLID	HALLNUMBER	HALLCAPACITY	TICKETID
1	1	201	180	3
2	2	202	200	1
3	3	203	220	5
4	4	204	160	2
5	5	205	250	7
6	6	206	190	4
7	7	207	300	9
8	8	208	210	6
9	9	209	230	10
10	10	210	260	8

Figure 37: Query Result of inserting Hall Table

- Theater Table

```
// Theater Table
INSERT INTO Theater VALUES (1,'Kumari Cinemas Civil Mall','Kathmandu',3);
INSERT INTO Theater VALUES (2,'Kumari Cinemas Lakeside','Pokhara',1);
INSERT INTO Theater VALUES (3,'Kumari Cinemas Janakpur','Janakpur',5);
INSERT INTO Theater VALUES (4,'Kumari Cinemas Butwal','Butwal',2);
INSERT INTO Theater VALUES (5,'Kumari Cinemas Surkhet','Surkhet',7);
INSERT INTO Theater VALUES (6,'Kumari Cinemas Tansen','Tansen',4);
INSERT INTO Theater VALUES (7,'Kumari Cinemas Rajbiraj','Rajbiraj',9);
INSERT INTO Theater VALUES (8,'Kumari Cinemas Damak','Damak',6);
INSERT INTO Theater VALUES (9,'Kumari Cinemas Tikapur','Tikapur',10);
INSERT INTO Theater VALUES (10,'Kumari Cinemas Gaur','Gaur',8);
```

Figure 38: Inserting data in Theater Table

Columns | Data | Model | Constraints | Grants | Statistics | Triggers | Flashback | Dependencies | Details | Partitions

Sort.. | Filter:

	THEATERID	THEATERNAME	THEATERCITY	TICKETID
1	1	Kumari Cinemas Civil Mall	Kathmandu	3
2	2	Kumari Cinemas Lakeside	Pokhara	1
3	3	Kumari Cinemas Janakpur	Janakpur	5
4	4	Kumari Cinemas Butwal	Butwal	2
5	5	Kumari Cinemas Surkhet	Surkhet	7
6	6	Kumari Cinemas Tansen	Tansen	4
7	7	Kumari Cinemas Rajbiraj	Rajbiraj	9
8	8	Kumari Cinemas Damak	Damak	6
9	9	Kumari Cinemas Tikapur	Tikapur	10
10	10	Kumari Cinemas Gaur	Gaur	8

Figure 39: Query Result of inserting Theater Table

7 Test Cases

7.1 Test Case for Basic Webform

- **User Table**

1. Inserting New in User Management

Table 1: Table 1: Test case for inserting new user

Test	Inserting new user in the user management webform
Action	Inserting the new user details in the user management web form.
Expected Result	The new added user details should be added to the user table.
Actual Result	After inserting the new user details, it is added in the user table.
Conclusion	Test is Successful.

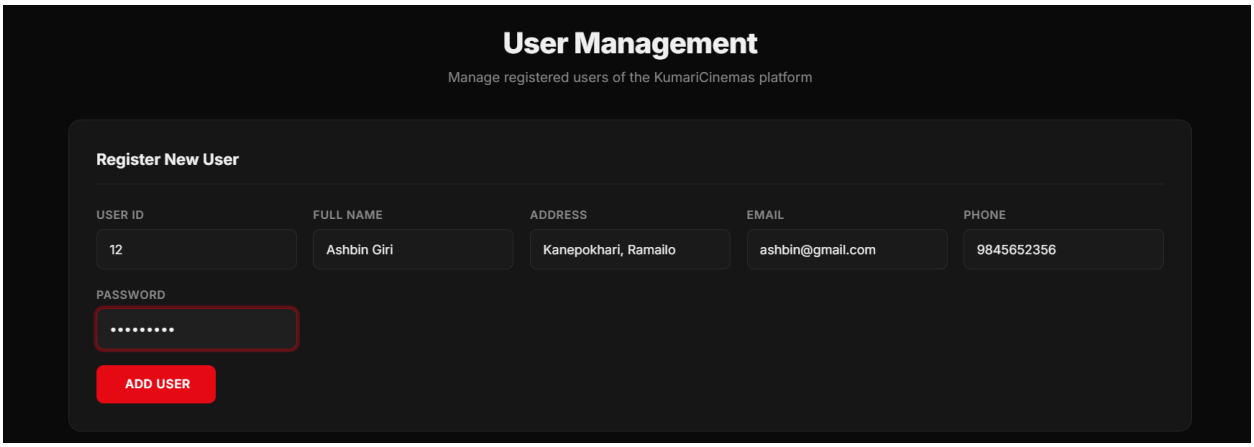
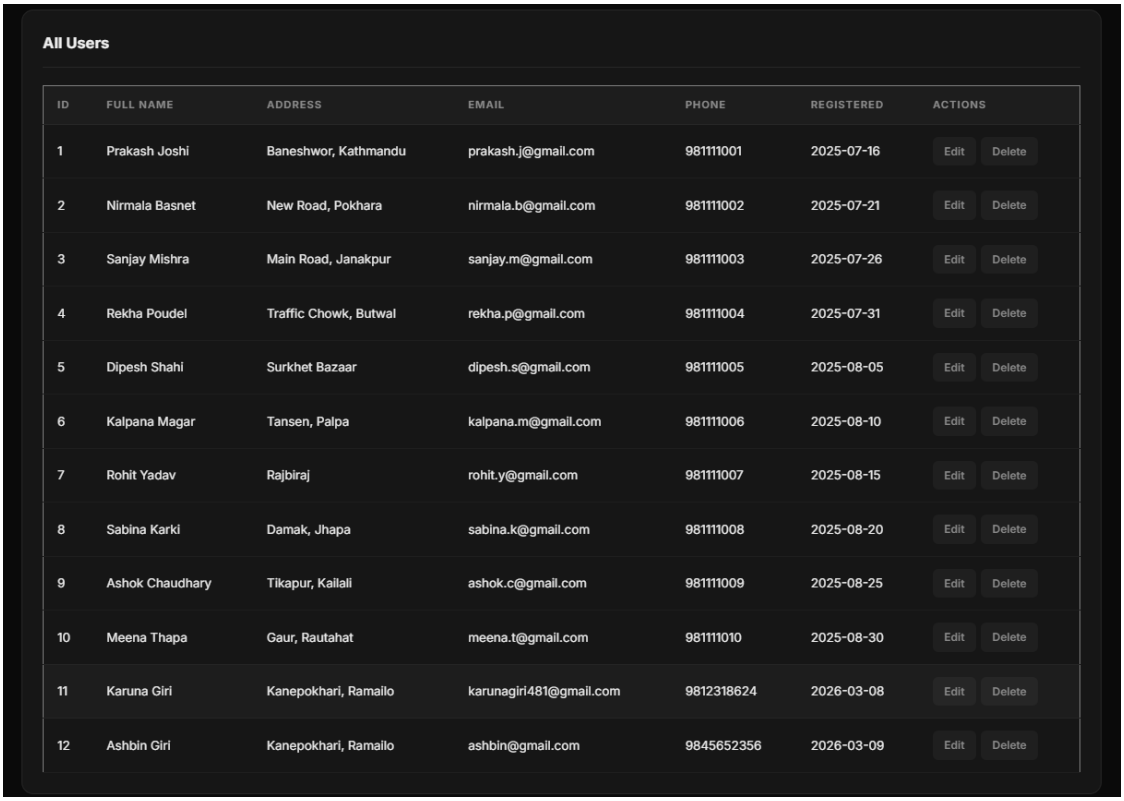


Figure 40: Inserting new User Details



All Users

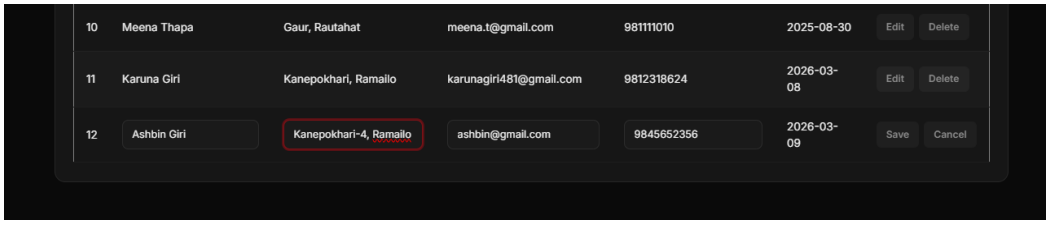
ID	FULL NAME	ADDRESS	EMAIL	PHONE	REGISTERED	ACTIONS
1	Prakash Joshi	Baneshwor, Kathmandu	prakash.j@gmail.com	981111001	2025-07-16	Edit Delete
2	Nirmala Basnet	New Road, Pokhara	nirmala.b@gmail.com	981111002	2025-07-21	Edit Delete
3	Sanjay Mishra	Main Road, Janakpur	sanjay.m@gmail.com	981111003	2025-07-26	Edit Delete
4	Rekha Poudel	Traffic Chowk, Butwal	rekha.p@gmail.com	981111004	2025-07-31	Edit Delete
5	Dipesh Shahi	Surkhet Bazaar	dipesh.s@gmail.com	981111005	2025-08-05	Edit Delete
6	Kalpana Magar	Tansen, Palpa	kalpana.m@gmail.com	981111006	2025-08-10	Edit Delete
7	Rohit Yadav	Rajbiraj	rohit.y@gmail.com	981111007	2025-08-15	Edit Delete
8	Sabina Karki	Damak, Jhapa	sabina.k@gmail.com	981111008	2025-08-20	Edit Delete
9	Ashok Chaudhary	Tikapur, Kailali	ashok.c@gmail.com	981111009	2025-08-25	Edit Delete
10	Meena Thapa	Gaur, Rautahat	meena.t@gmail.com	981111010	2025-08-30	Edit Delete
11	Karuna Giri	Kanepokhari, Ramailo	karunagiri481@gmail.com	9812318624	2026-03-08	Edit Delete
12	Ashbin Giri	Kanepokhari, Ramailo	ashbin@gmail.com	9845652356	2026-03-09	Edit Delete

Figure 41: Result of Successfully added in the User Table

2. Editing User Data in User Management

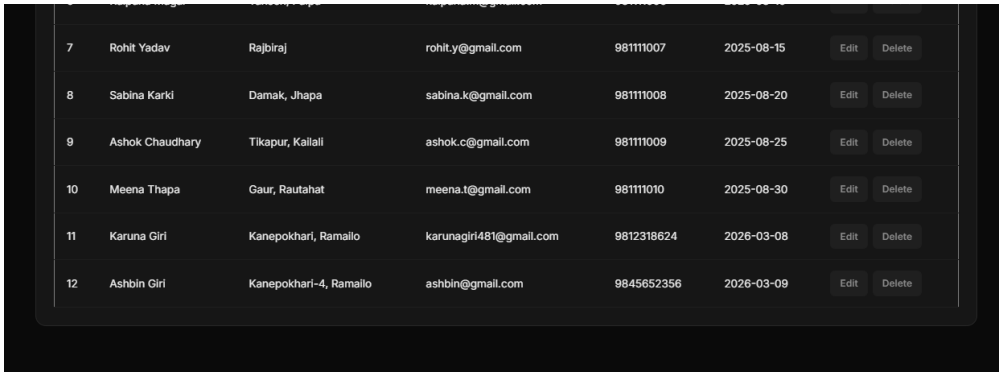
Table 2: Test Case of updating user details

Test	Updating user details of user table in the user management webform.
Action	Updating the user details of user table.
Expected Result	After editing the user details, it should be updated in the user table.
Actual Result	The user data has been successfully updated in the user table.
Conclusion	Test is Successful.



10	Meena Thapa	Gaur, Rautahat	meena.t@gmail.com	981111010	2025-08-30	Edit	Delete
11	Karuna Giri	Kanepokhari, Ramailo	karunagiri481@gmail.com	9812318624	2026-03-08	Edit	Delete
12	Ashbin Giri	Kanepokhari-4, Ramailo	ashbin@gmail.com	9845652356	2026-03-09	Save	Cancel

Figure 42: Updating exiting user details



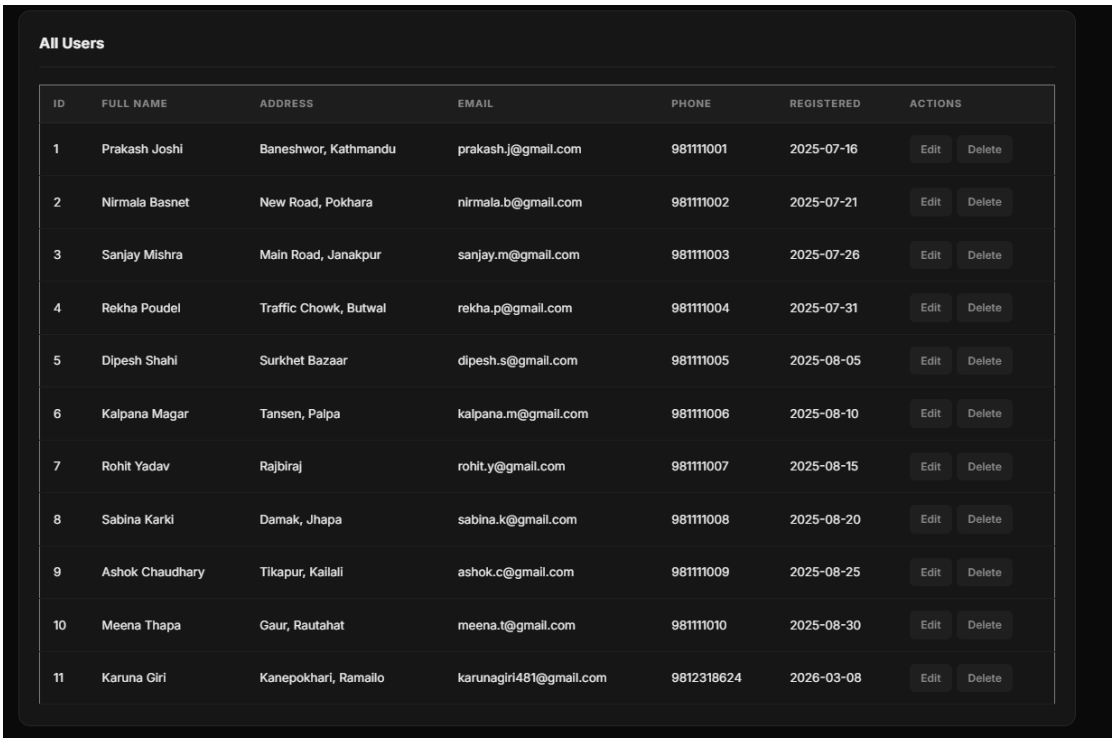
7	Rohit Yadav	Rajbiraj	rohit.y@gmail.com	981111007	2025-08-15	Edit	Delete
8	Sabina Karki	Damak, Jhapa	sabina.k@gmail.com	981111008	2025-08-20	Edit	Delete
9	Ashok Chaudhary	Tikapur, Kailali	ashok.c@gmail.com	981111009	2025-08-25	Edit	Delete
10	Meena Thapa	Gaur, Rautahat	meena.t@gmail.com	981111010	2025-08-30	Edit	Delete
11	Karuna Giri	Kanepokhari, Ramailo	karunagiri481@gmail.com	9812318624	2026-03-08	Edit	Delete
12	Ashbin Giri	Kanepokhari-4, Ramailo	ashbin@gmail.com	9845652356	2026-03-09	Edit	Delete

Figure 43: Successfully updated in the user table

3. Deleting User Data in User Management

Table 3: Test Case of deleting user

Test	Deleting user details of user table in the user management webform.
Action	Deleting the user details of user table.
Expected Result	After deleting the user details, it should be removed from the user table.
Actual Result	The user data has been successfully removed from the user table.
Conclusion	Test is Successful.



All Users

ID	FULL NAME	ADDRESS	EMAIL	PHONE	REGISTERED	ACTIONS
1	Prakash Joshi	Baneshwor, Kathmandu	prakash.j@gmail.com	981111001	2025-07-16	Edit Delete
2	Nirmala Basnet	New Road, Pokhara	nirmala.b@gmail.com	981111002	2025-07-21	Edit Delete
3	Sanjay Mishra	Main Road, Janakpur	sanjay.m@gmail.com	981111003	2025-07-26	Edit Delete
4	Rekha Poudel	Traffic Chowk, Butwal	rekha.p@gmail.com	981111004	2025-07-31	Edit Delete
5	Dipesh Shahi	Surkhet Bazaar	dipesh.s@gmail.com	981111005	2025-08-05	Edit Delete
6	Kalpana Magar	Tansen, Palpa	kalpana.m@gmail.com	981111006	2025-08-10	Edit Delete
7	Rohit Yadav	Rajbiraj	rohit.y@gmail.com	981111007	2025-08-15	Edit Delete
8	Sabina Karki	Damak, Jhapa	sabina.k@gmail.com	981111008	2025-08-20	Edit Delete
9	Ashok Chaudhary	Tikapur, Kailali	ashok.c@gmail.com	981111009	2025-08-25	Edit Delete
10	Meena Thapa	Gaur, Rautahat	meena.t@gmail.com	981111010	2025-08-30	Edit Delete
11	Karuna Giri	Kanepokhari, Ramailo	karunagiri481@gmail.com	9812318624	2026-03-08	Edit Delete

Figure 44: Successfully removed form user table

- **Theatre Table**

4. Inserting Theater Data in Theatre Management

Table 4: Test case for inserting new theatre data

Test	Inserting new theater data in the theater management webform.
Action	Inserting the new theater details in the theater management web form.
Expected Result	The new added theater details should be added to the theater table.
Actual Result	After inserting the new theater details, it is added in the theater table.
Conclusion	Test is Successful.

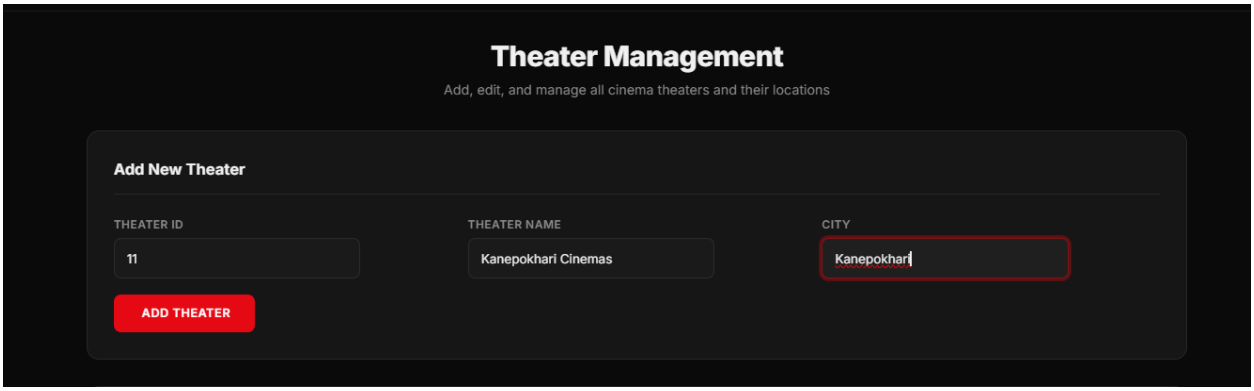


Figure 45: Inserting new Theater Details

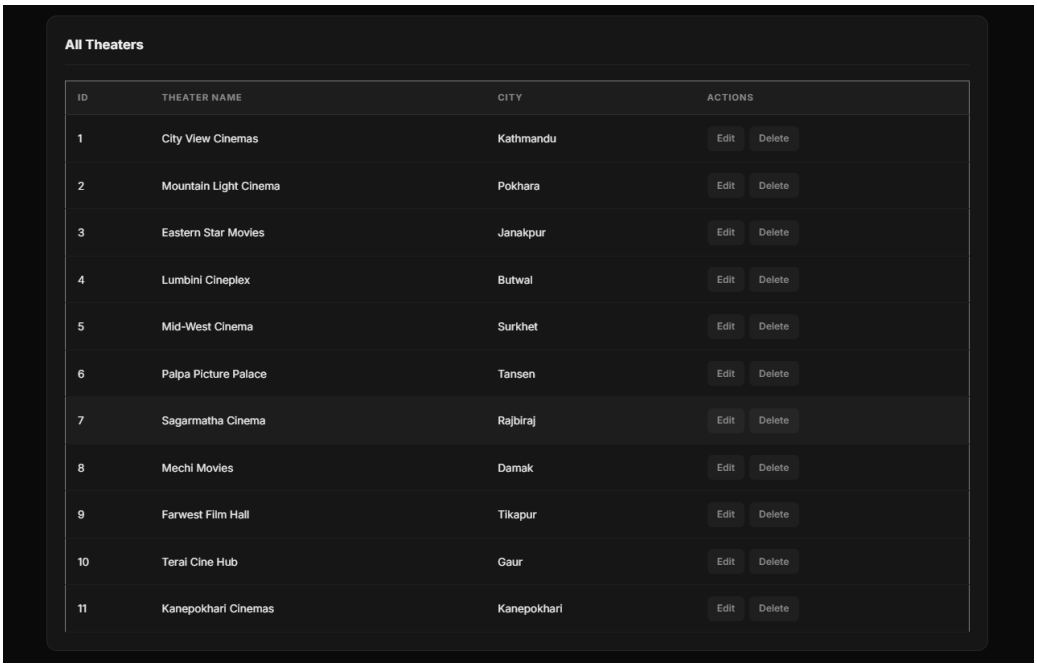


Figure 46: Result of Successfully added in the Theater Table

5. Editing Theater Data in Theatre Management

Table 5: Test Case of updating theatre details

Test	Updating theatre details in the Theater Management.
Action	Updating the user details in the theatre management table.
Expected Result	After editing the theatre details, it should be updated in the theatre table.
Actual Result	The data has been successfully updated in the theatre table.
Conclusion	Test is Successful.

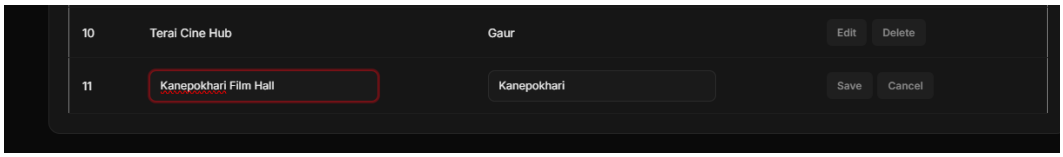


Figure 47: Updating exiting theatre details

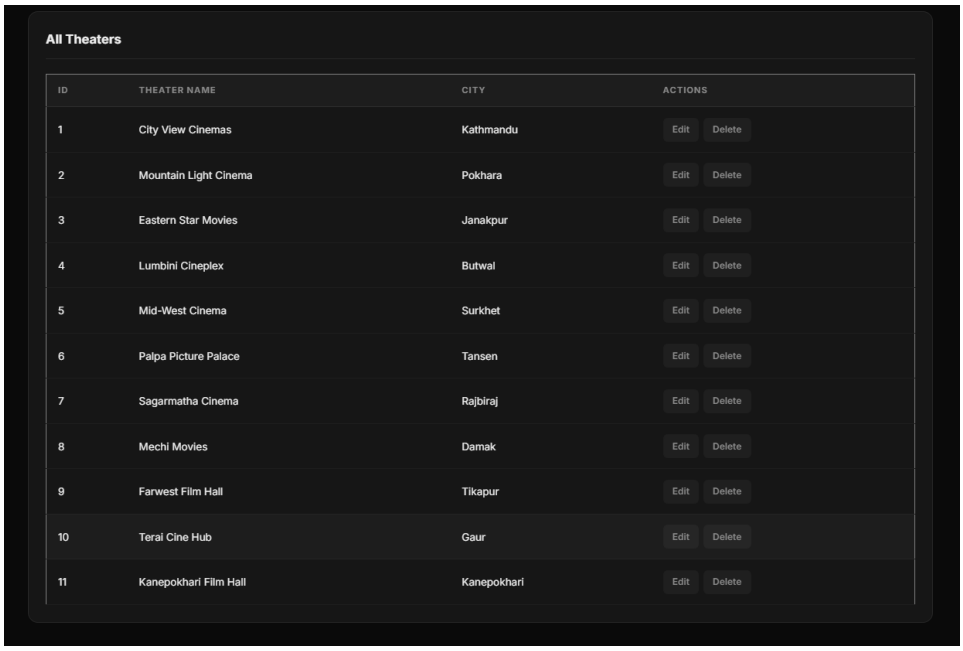


Figure 48: Successfully updated in the theatre table

6. Deleting Theatre Data in Theatre Management

Table 6: Test Case of deleting theatre data

Test	Deleting theatre details of theatre table in the theatre management webform.
Action	Deleting the theatre details of theatre table.
Expected Result	After deleting the theatre details, it should be removed from the table.
Actual Result	The theatre data has been successfully removed from the table.
Conclusion	Test is Successful.

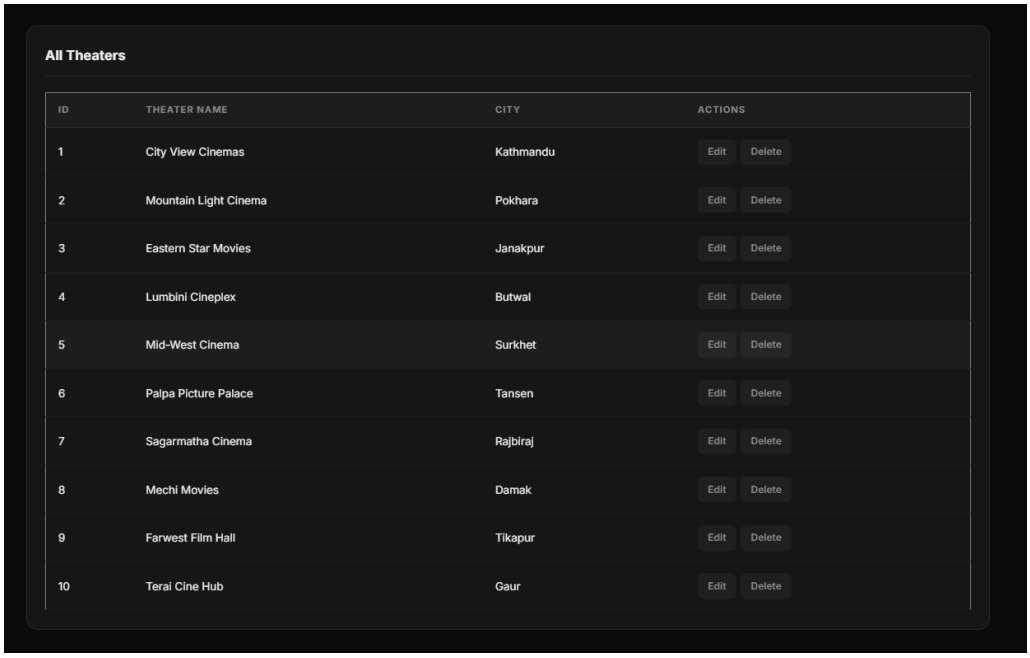


Figure 49: Successfully removed form theatre table

- **Movies Table**

1. Inserting Movie Data in Movie Management

Table 7: Test case for inserting new movie data

Test	Inserting new movie data in the movie management webform.
Action	Inserting the new movie details in the movie management web form.
Expected Result	The new added movie details should be added to the movie table.
Actual Result	After inserting the new movie details, it is added in the movie table.
Conclusion	Test is Successful.

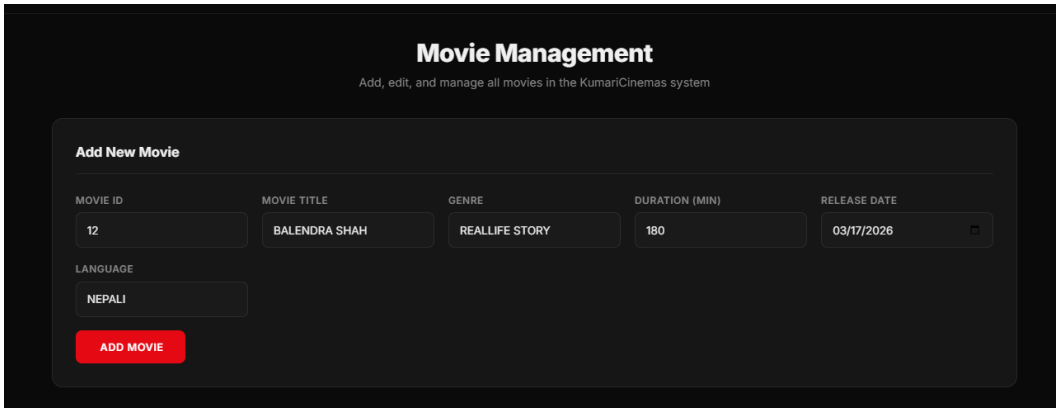
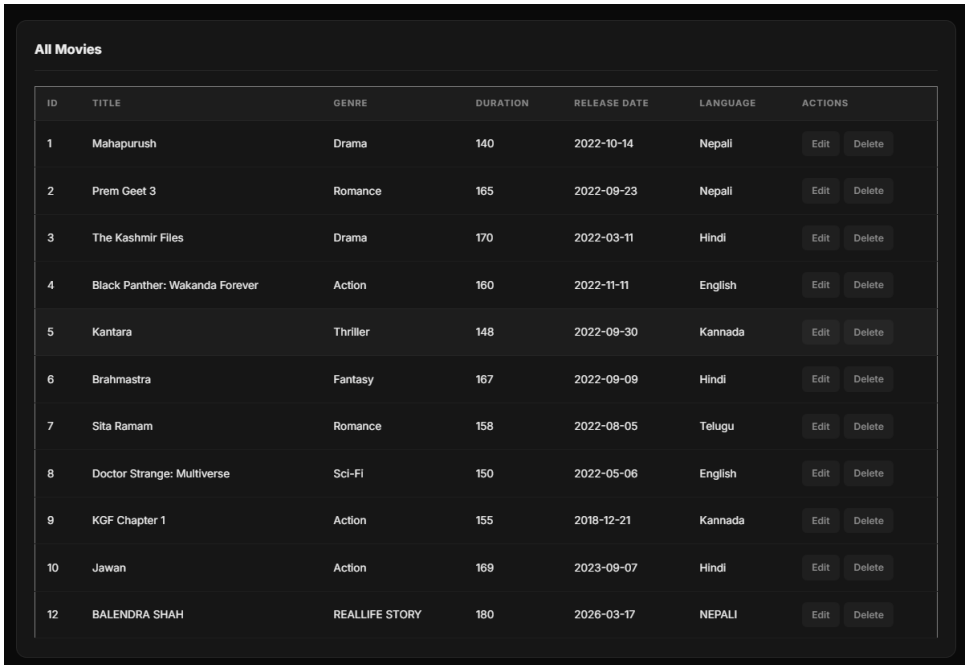


Figure 50: Inserting new Movie Details



All Movies

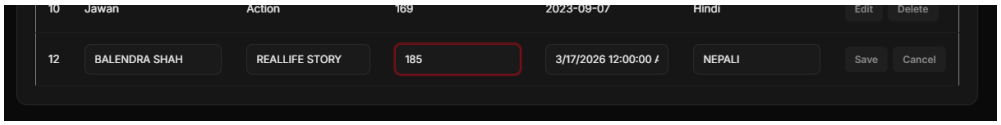
ID	TITLE	GENRE	DURATION	RELEASE DATE	LANGUAGE	ACTIONS
1	Mahapurush	Drama	140	2022-10-14	Nepali	Edit Delete
2	Prem Geet 3	Romance	165	2022-09-23	Nepali	Edit Delete
3	The Kashmir Files	Drama	170	2022-03-11	Hindi	Edit Delete
4	Black Panther: Wakanda Forever	Action	160	2022-11-11	English	Edit Delete
5	Kantara	Thriller	148	2022-09-30	Kannada	Edit Delete
6	Brahmastra	Fantasy	167	2022-09-09	Hindi	Edit Delete
7	Sita Ramam	Romance	158	2022-08-05	Telugu	Edit Delete
8	Doctor Strange: Multiverse	Sci-Fi	150	2022-05-06	English	Edit Delete
9	KGF Chapter 1	Action	155	2018-12-21	Kannada	Edit Delete
10	Jawan	Action	169	2023-09-07	Hindi	Edit Delete
12	BALENDRA SHAH	REALLIFE STORY	180	2026-03-17	NEPALI	Edit Delete

Figure 51: Result of Successfully added in the Movie Table

2. Editing Movie Data in Movie Management

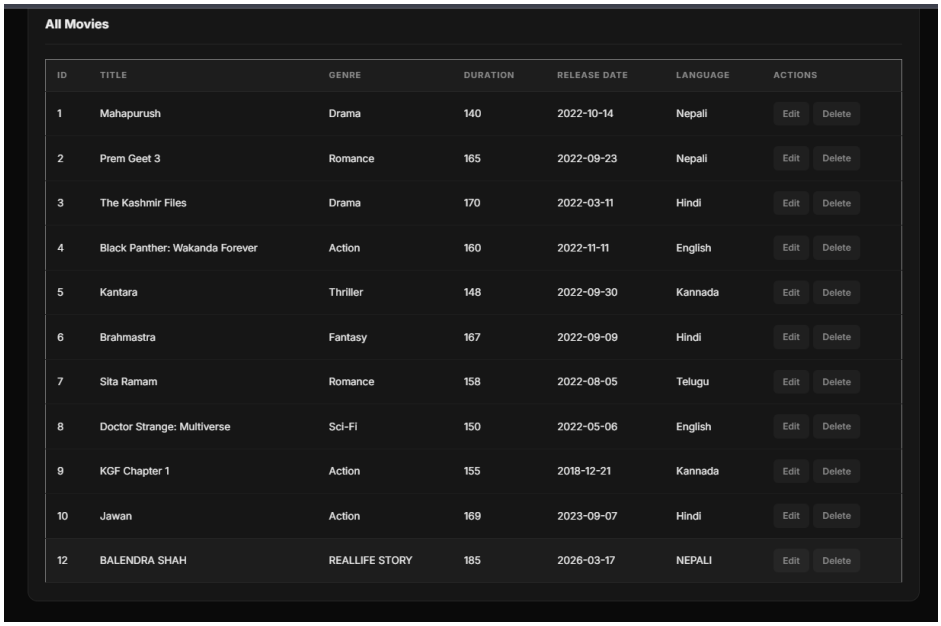
Table 8: Test Case of updating movie details

Test	Updating movie details in the movie management.
Action	Updating the movie details in the movie management table.
Expected Result	After editing the movie details, it should be updated in the movie table.
Actual Result	The data has been successfully updated in the movie table.
Conclusion	Test is Successful.



10	Jawan	Action	169	2023-09-07	Hindi	Edit Delete
12	BALENDRA SHAH	REALLIFE STORY	185	3/17/2026 12:00:00 A	NEPALI	Save Cancel

Figure 52: Updating exiting movie details



All Movies

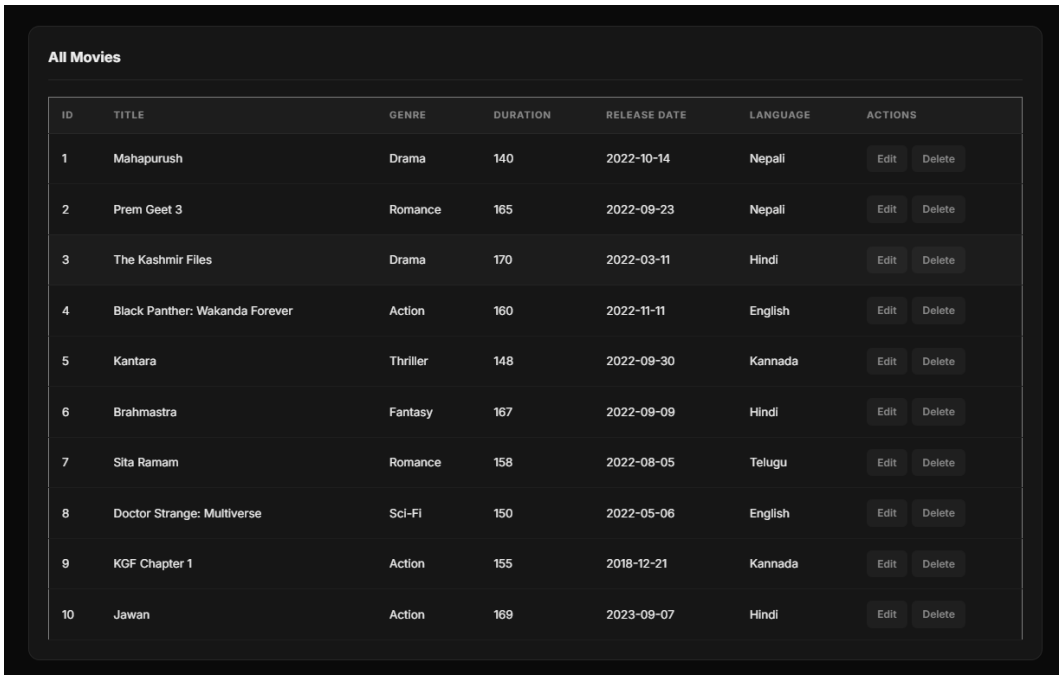
ID	TITLE	GENRE	DURATION	RELEASE DATE	LANGUAGE	ACTIONS
1	Mahapurush	Drama	140	2022-10-14	Nepali	Edit Delete
2	Prem Geet 3	Romance	165	2022-09-23	Nepali	Edit Delete
3	The Kashmir Files	Drama	170	2022-03-11	Hindi	Edit Delete
4	Black Panther: Wakanda Forever	Action	160	2022-11-11	English	Edit Delete
5	Kantara	Thriller	148	2022-09-30	Kannada	Edit Delete
6	Brahmastra	Fantasy	167	2022-09-09	Hindi	Edit Delete
7	Sita Ramam	Romance	158	2022-08-05	Telugu	Edit Delete
8	Doctor Strange: Multiverse	Sci-Fi	150	2022-05-06	English	Edit Delete
9	KGF Chapter 1	Action	155	2018-12-21	Kannada	Edit Delete
10	Jawan	Action	169	2023-09-07	Hindi	Edit Delete
12	BALENDRA SHAH	REALLIFE STORY	185	2026-03-17	NEPALI	Edit Delete

Figure 53: Successfully updated in the movie table

3. Deleting Movie Data in Movie Management

Table 9: Test Case of deleting movie data

Test	Deleting movie details of movie table in the movie management webform.
Action	Deleting the movie details of movie table.
Expected Result	After deleting the movie details, it should be removed from the table.
Actual Result	The movie data has been successfully removed from the table.
Conclusion	Test is Successful.



All Movies

ID	TITLE	GENRE	DURATION	RELEASE DATE	LANGUAGE	ACTIONS
1	Mahapurush	Drama	140	2022-10-14	Nepali	Edit Delete
2	Prem Geet 3	Romance	165	2022-09-23	Nepali	Edit Delete
3	The Kashmir Files	Drama	170	2022-03-11	Hindi	Edit Delete
4	Black Panther: Wakanda Forever	Action	160	2022-11-11	English	Edit Delete
5	Kantara	Thriller	148	2022-09-30	Kannada	Edit Delete
6	Brahmastra	Fantasy	167	2022-09-09	Hindi	Edit Delete
7	Sita Ramam	Romance	158	2022-08-05	Telugu	Edit Delete
8	Doctor Strange: Multiverse	Sci-Fi	150	2022-05-06	English	Edit Delete
9	KGF Chapter 1	Action	155	2018-12-21	Kannada	Edit Delete
10	Jawan	Action	169	2023-09-07	Hindi	Edit Delete

Figure 54: Successfully removed form movie table

- **ShowTime Table**

1. Inserting ShowTime Data in ShowTime Management

Table 10: Test case for inserting new ShowTime data

Test	Inserting new showtime data in the showtime management webform.
Action	Inserting the new showtime details in the showtime management web form.
Expected Result	The new added showtime details should be added to the showtime table.
Actual Result	After inserting the new showtime details, it is added in the showtime table.
Conclusion	Test is Successful.

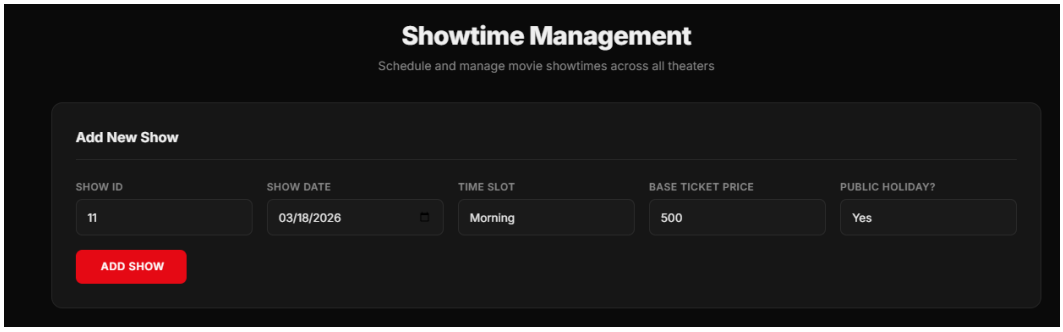


Figure 55: Inserting new showtime Details

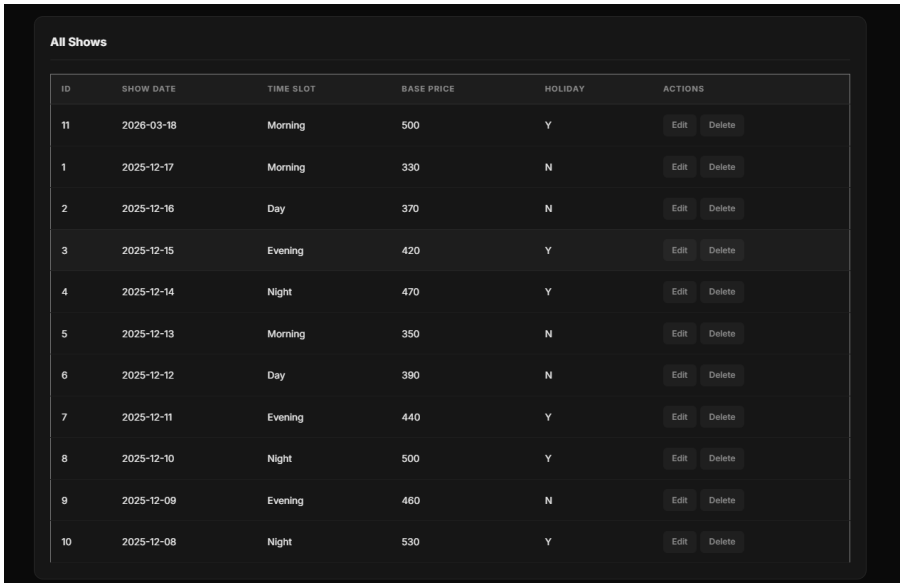


Figure 56: Result of Successfully added in the Showtime Table

2. Editing ShowTime Data in Movie Management

Table 11: Test Case of updating showtime details

Test	Updating showtime details in the showtime management.
Action	Updating the showtime details in the showtime management table.
Expected Result	After editing the showtime details, it should be updated in the showtime table.
Actual Result	The data has been successfully updated in the showtime table.
Conclusion	Test is Successful.

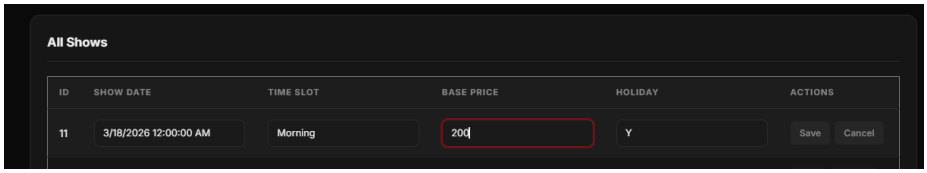
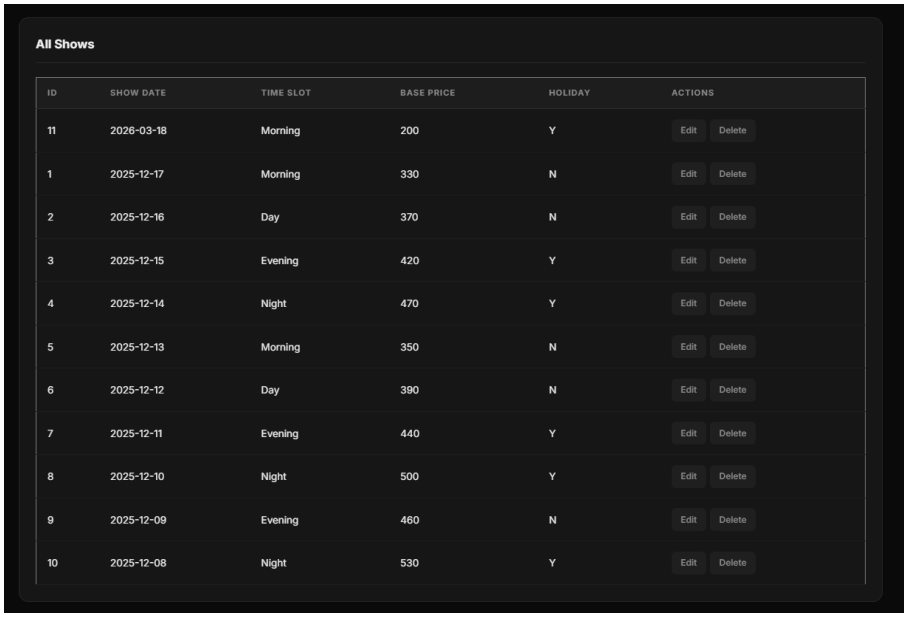


Figure 57: Updating exiting showtime details



All Shows

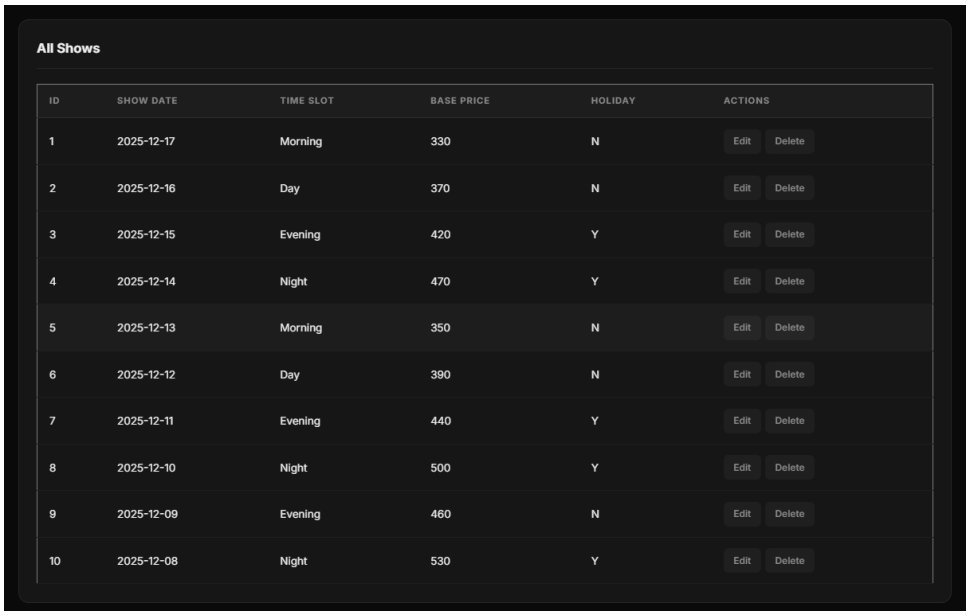
ID	SHOW DATE	TIME SLOT	BASE PRICE	HOLIDAY	ACTIONS
11	2026-03-18	Morning	200	Y	Edit Delete
1	2025-12-17	Morning	330	N	Edit Delete
2	2025-12-16	Day	370	N	Edit Delete
3	2025-12-15	Evening	420	Y	Edit Delete
4	2025-12-14	Night	470	Y	Edit Delete
5	2025-12-13	Morning	350	N	Edit Delete
6	2025-12-12	Day	390	N	Edit Delete
7	2025-12-11	Evening	440	Y	Edit Delete
8	2025-12-10	Night	500	Y	Edit Delete
9	2025-12-09	Evening	460	N	Edit Delete
10	2025-12-08	Night	530	Y	Edit Delete

Figure 58: Successfully updated in the showtime table

3. Deleting ShowTime Data in Movie Management

Table 12: Test Case of deleting showtime data

Test	Deleting movie details of showtime table in the showtime management webform.
Action	Deleting the showtime details of showtime table.
Expected Result	After deleting the showtime details, it should be removed from the table.
Actual Result	The showtime data has been successfully removed from the table.
Conclusion	Test is Successful.



All Shows

ID	SHOW DATE	TIME SLOT	BASE PRICE	HOLIDAY	ACTIONS
1	2025-12-17	Morning	330	N	Edit Delete
2	2025-12-16	Day	370	N	Edit Delete
3	2025-12-15	Evening	420	Y	Edit Delete
4	2025-12-14	Night	470	Y	Edit Delete
5	2025-12-13	Morning	350	N	Edit Delete
6	2025-12-12	Day	390	N	Edit Delete
7	2025-12-11	Evening	440	Y	Edit Delete
8	2025-12-10	Night	500	Y	Edit Delete
9	2025-12-09	Evening	460	N	Edit Delete
10	2025-12-08	Night	530	Y	Edit Delete

Figure 59: Successfully removed form showtime table

7.2 Test Case for Complex Forms

- **User Tickets**

1. Test Case for User Ticket

Table 13: Test Case of User Tickets

Test	Testing the user ticket report to verify that the system displays user information and purchased tickets in the past six months.
Action	Select a specific user ID and access the specific user information along with the tickets purchased over the last six months.
Expected Result	A The system should display the information of the selected user and only those tickets that were bought in the last six months.
Actual Result	The system successfully shows the user details and the tickets within the period of six months.
Conclusion	Test is Successful.

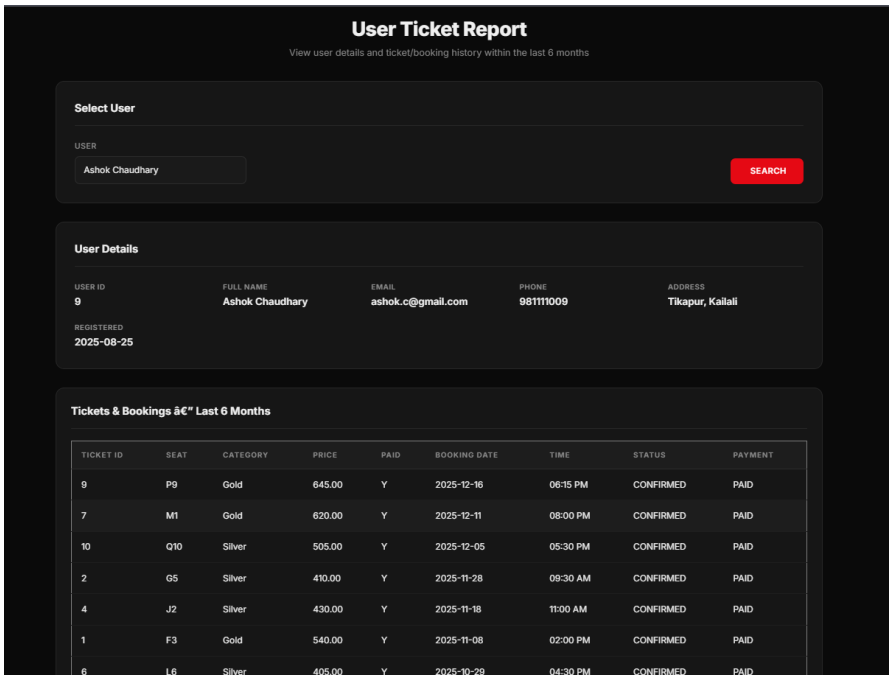


Figure 60: Figure of User Tickets

- **TheaterHallCity Movie**

2. Test Case for TheaterHallCity Movie

Table 14: Test Case of TheaterHallCity Movie

Test	The TheaterHallCity Movie Report can be tested to display the movie details and corresponding show times.
Action	Select a specific TheaterHallCity and view the movies scheduled along with their showtimes.
Expected Result	The system should display the selected TheaterHallCity and their time of showing.
Actual Result	The movies and their showtimes of the selected TheaterHallCity are displayed successfully
Conclusion	Test is Successful.

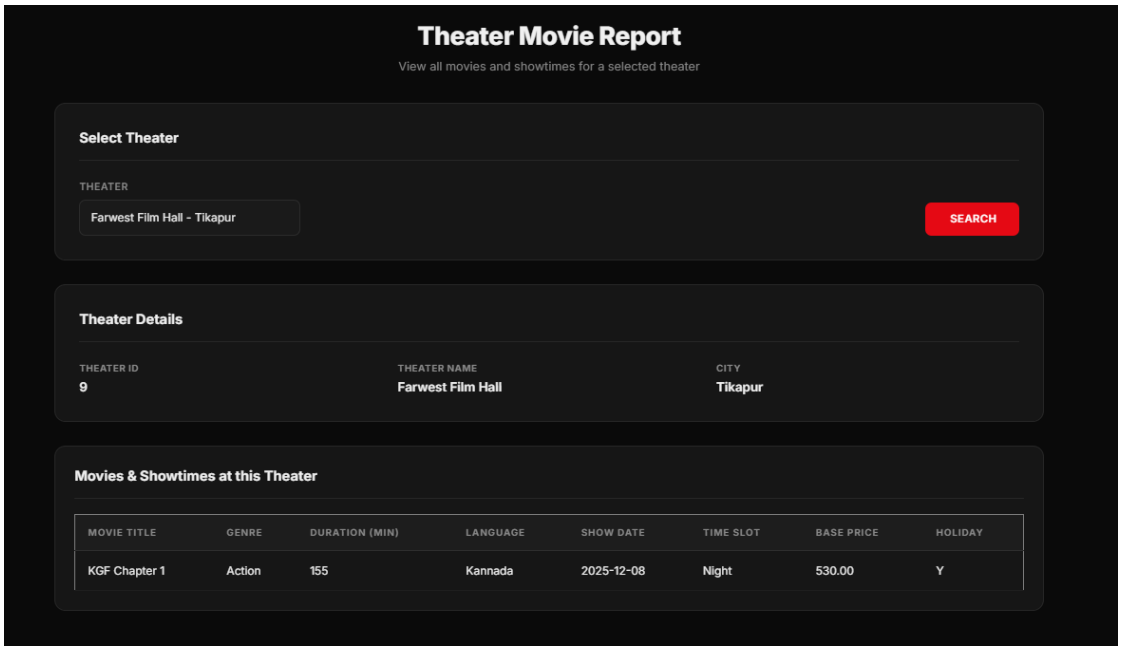


Figure 61: Figure of Theater Movie

- **Movie TheaterCityHallOccupancy Performer**

3. Test Case for Movie TheatreCityHallOccupancy Performer

Table 15: Test Case of Movie TheatreCityHallOccupancy Performer

Test	Testing the report that identifies the top three TheatreCityHall on the basis of movie seats occupation.
Action	Select a movie and check the first three halls in terms of occupancy by paying tickets.
Expected Result	The system should display the TheaterCityHalls according to the highest percent of place attendance according to the paid tickets.
Actual Result	The system successfully displays the halls that have the highest occupancy percentages.
Conclusion	Test is Successful.

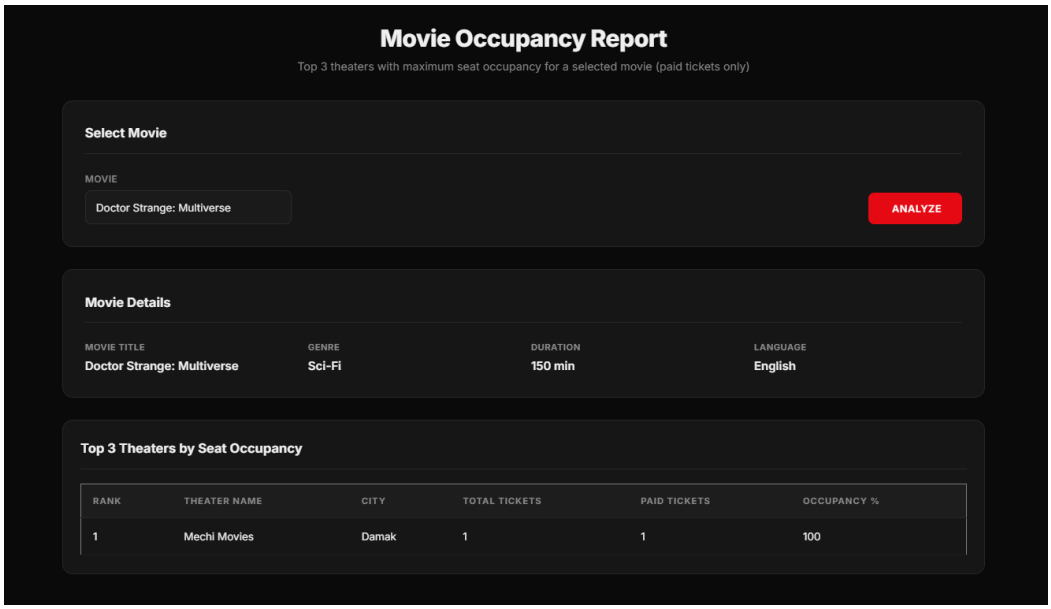


Figure 62: Figure of Movie TheatreCityHall Occupancy performer

8 Conclusion

The project designs a well-structured relational database of KumariCinemas that is responsive to the operational requirements of multi-cinemas ticketing environment. The report demonstrates how the problem of Data fragmentation, excessive operational expenses and constraints of current ticketing practices can be resolved with the implementation of a centralized database storage which constantly records the data regarding movies, customers, halls, show time and purchasing tickets. The design is built on a certain contextual basis of the operation of cinema in Nepal and therefore the work reflects an insight into both the technical and business needs.

In the design-based consideration, the work exhibits a good understanding of fundamental concepts in database design by taking the original design of the database to a final, fully normalized design. The concept of UNF, 1NF, 2NF and 3NF is connected to the UNF discussion of the data of KumariCinemas where large and unstructured table is deconstructed with smaller, subject-oriented tables that have well-defined primary and foreign keys. The result of this development is an ultimate database design in which elimination of redundancy, diminution of anomalies and the portrayal of the existence of entities (movies, theaters and tickets) among themselves is clearly conveyed.

The implementation part adds to the quality of the design by converting the normalized schema into an Oracle SQL Developer environment consisting of relevant CREATE TABLE statements and sample data. This is a working action that proves that the logical design is possible to be converted into the working database that could serve to perform cinema ticketing functions including various halls, session time and dynamic pricing. The analytical, design and implementation parts in general give a sound base to create a new web-based ticketing application of the future and show-twelve the efficient implementation of the database theory to a practical situation.

9 References

Alice Gomstyn, A.J. (2025) *What is database normalization?* [Online]. Available from: https://www.ibm.com/think/topics/database-normalization [Accessed 23 December 2025].

Dancuk, M. (2021) *What is Database Normalization?* [Online]. Available from: https://phoenixnap.com/kb/database-normalization [Accessed 23 December 2025].

dbdiagram.io. (2025) *Introduction* [Online]. Available from: https://docs.dbdiagram.io/ [Accessed 22 December 2025].

lucidchart. (2025) *Database Structure and Design Tutorial* [Online]. Available from: https://www.lucidchart.com/pages/tutorial/database-design-and-structure [Accessed 23 December 2025].

oracle. (2025) *SQL Developer* [Online]. Available from: https://www.oracle.com/database/sqldeveloper/ [Accessed 22 December 2025].